CAN THO UNIVERSITY

COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

FACULTY OF SOFTWARE ENGINEERING



GRADUATION THESIS
BACHELOR OF ENGINEERING IN

SOFTWARE ENGINEERING
(HIGH-QUALITY PROGRAM)

RESIDUE-LEVEL DEGRON MUTATION EFFECT PREDICTION AND ANALYSIS TOOL

Student: Nguyen Thanh Trong

Student ID: B2305615

Class: 2023 – 2027 (K49)

Advisor: Ho Quang Thai, Ph.D.

Can Tho, [MM/YYYY]

CAN THO UNIVERSITY

COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

FACULTY OF SOFTWARE ENGINEERING



GRADUATION THESIS
BACHELOR OF ENGINEERING IN
SOFTWARE ENGINEERING
(HIGH-QUALITY PROGRAM)

RESIDUE-LEVEL DEGRON MUTATION EFFECT PREDICTION AND ANALYSIS TOOL

Student:

Nguyen Thanh Trong

Student ID: B2305615

Class: DI2396F1, K49

Advisor:

Ho Quang Thai, Ph.D.

Can Tho, [MM/YYYY]

CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

SOCIALIST REPUBLIC OF VIETNAM
Independence – Freedom – Happiness

THESIS REVISION CONFIRMATION IN ACCORDANCE WITH THE COMMITTEE'S COMMENTS

Thesis Title: RESIDUE-LEVEL DEGRON MUTATION EFFECT PREDICTION AND ANALYSIS TOOL

Student: Nguyen Thanh Trong Student ID: B2305615

Class Code: DI2396F1

The thesis was presented before the Thesis Examination Committee of the Software Engineering Program.

Members of the Examination Committee:

- [Name], Ph.D.
- [Name], Ph.D.
- [Name], Ph.D.

Reporting date: [Month DD, YYYY]

The thesis has been revised in accordance with the Committee's comments.

Can Tho, ……

Supervisor

Ho Quang Thai, Ph.D.

COMMENTS OF THE ADVISOR

. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .

Can Tho, ……

Advisor

Ho Quang Thai, Ph.D.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my advisor, Dr. Ho Quang Thai, for his guidance throughout this thesis. I also thank the lecturers of the Faculty of Software Engineering, College of Information and Communication Technology, Can Tho University, for the knowledge they gave me, and my family and friends for their support. **[PENDING: review and personalise before submission.]**

Can Tho, ……

Student

Nguyen Thanh Trong

COMMITMENT TO RESULTS

I hereby certify that the graduation thesis titled "RESIDUE-LEVEL DEGRON MUTATION EFFECT PREDICTION AND ANALYSIS TOOL" is the result of my own research and has not been submitted for any other degree or qualification.

All references used in the thesis are clearly cited, including the author's name, the title of the work, and the time and place of publication. I accept full responsibility for any unauthorized copying, violations of academic regulations, or instances of academic dishonesty.

Can Tho, ……

Student

Nguyen Thanh Trong

ABSTRACT

A degron is a sequence element that marks a protein for selective degradation, and a single amino acid substitution within it can change its activity. Experiments score whole peptides, while measuring every substitution is costly. This thesis studies how to predict the effect of single amino acid substitutions in degron peptides at residue level from experimental stability data, evaluates generalisation with protein-cluster-aware splits against ridge, PAP-difference and Degformer baselines, and builds a prototype local React/FastAPI tool whose first-release scope includes predicted effect matrices, substitution ranking, peptide batches and whole-protein tiling, ClinVar/reference annotations, experimental-data comparison, persistent projects/history and export with provenance. These are software requirements; the expanded workflows remain pending implementation and verification. **[PENDING: main results and limitations, to be written when the work is complete.]**

Keywords: degron, protein stability, mutation effect prediction, ubiquitin–proteasome system, machine learning.

TÓM TẮT

Degron là đoạn trình tự đánh dấu protein để bị phân hủy có chọn lọc; một phép thay thế amino acid trong degron có thể làm thay đổi hoạt tính này. Thực nghiệm thường chấm điểm cả peptide, trong khi đo mọi phép thay thế rất tốn kém. Luận văn nghiên cứu cách dự đoán hiệu ứng của phép thay thế đơn amino acid trong peptide degron ở mức residue từ dữ liệu độ bền protein thực nghiệm, đánh giá khả năng tổng quát hóa khi chia dữ liệu theo cụm protein so với các đối chứng ridge, chênh lệch điểm PAP và Degformer, đồng thời xây dựng ứng dụng local bằng React/FastAPI. Phạm vi bản đầu gồm ma trận hiệu ứng, xếp hạng phép thay thế, xử lý theo lô peptide và chia cửa sổ trên toàn protein, chú giải ClinVar/nguồn tham chiếu, nhập dữ liệu đo để đối chiếu, lưu dự án/lịch sử và xuất kết quả kèm nguồn gốc. Đây là yêu cầu phần mềm; các luồng mở rộng còn chờ triển khai và kiểm chứng. **[CHỜ BỔ SUNG: kết quả chính và giới hạn, viết khi hoàn thành công việc.]**

Từ khóa: degron, độ bền protein, dự đoán hiệu ứng đột biến, hệ ubiquitin–proteasome, học máy.

TABLE OF CONTENTS

THESIS REVISION CONFIRMATION i

COMMENTS OF THE ADVISOR ii

ACKNOWLEDGEMENTS iii

COMMITMENT TO RESULTS iv

ABSTRACT v

TÓM TẮT vi

TABLE OF CONTENTS vii

LIST OF FIGURES xii

LIST OF TABLES xiii

GLOSSARY AND ABBREVIATIONS xv

PART 1. INTRODUCTION 1

1. Problem Statement 1

2. History of Approaches to the Problem 1

2.1. International Context 1

2.2. Domestic Context 1

3. Objectives 2

3.1. General Objective 2

3.2. Specific Objectives 2

4. Research Subject and Scope 2

4.1. Research Subject 2

4.2. Research Scope 3

5. Research Process and Technologies Used 3

5.1. Research Process 3

5.2. Technologies Used 3

5.3. Development and Supporting Tools 4

6. Main Contributions 4
7. Structure of the Thesis 4

PART 2. CONTENT 5

CHAPTER 1: PROBLEM DESCRIPTION 5
1.1. Detailed Problem Description 5
1.2. Functions of the System 5
1.2.1. Predict the Effect Matrix 6
1.2.2. Rank Substitutions 6
1.2.3. Export Results with Provenance 7
1.2.4. Select a Protein and Tiles 8
1.2.5. Compare with Measured Data 8
1.2.6. View ClinVar and Reference Annotations 9
1.2.7. View Model and Data Provenance 10
1.2.8. Analyse Peptide Batches or a Whole Protein 10
1.2.9. Import Experimental Observations 11
1.2.10. Manage Projects and Analysis History 12
1.3. Non-Functional Requirements 13
1.3.1. Performance Requirements 13
1.3.2. Information Safety Requirements 13
1.3.3. Security Requirements 13
1.3.4. Reliability Requirements 13
1.3.5. Maintainability Requirements 14
1.3.6. Usability and Accessibility Requirements 14
1.4. Implementation and Design Constraints 14
1.4.1. Implementation Constraints 14
1.4.2. Design Constraints 14

CHAPTER 2: THEORETICAL BACKGROUND 15
2.1. Domain Background 15
2.1.1. Proteins, Amino Acids and Residues 15
2.1.2. The Ubiquitin–Proteasome System and Degrons 15
2.1.3. Measuring Degron Activity 15

2.2. Methods and Models 16
2.2.1. Problem Formulation 16
2.2.2. Baseline and Reference Models 16
2.2.3. Transfer Learning 16
2.3. Technologies and Tools 16
2.4. Evaluation Methodology 17
2.4.1. Splitting by Protein Cluster 17
2.4.2. Metrics 17
2.4.3. Statistical Comparison 17

CHAPTER 3: SOLUTION DESIGN AND IMPLEMENTATION 18
3.1. System Overview 18
3.1.1. Product Context 18
3.1.2. Functional Overview 18
3.2. System Architecture 18
3.2.1. Architectural Components 18
3.2.2. Interaction Between Components 19
3.2.3. Design Rationale 19
3.3. Data Design 20
3.3.1. Data Sources and Preprocessing 20
3.3.2. Data Model 20
3.3.3. Data Dictionary 20
3.3.4. Relationships Between Entities 20
3.4. Method and Model Design 21
3.5. Functional Design 21
3.5.1. Batch and Protein Processing 22
3.5.2. Import and Comparison 22
3.5.3. Annotations and Project Screens 22
3.6. Local Execution View 23

CHAPTER 4: TESTING AND EVALUATION 24
4.1. Testing Objectives 24
4.2. Experimental Evaluation 24
4.2.1. Experimental Setup 24

4.2.2. Evaluation Metrics 26
4.2.3. Results 27
4.2.3.1. Direct Learning on the Saturation Matrix 27
4.2.3.2. Published Degformer Checkpoint and Retrained Architecture 28
4.2.3.3. Interaction Designs 29
4.2.3.4. Learning Curve and Relaxing the Degron Threshold 29
4.2.3.5. Number of Clusters or Coverage of the Distribution 31
4.2.3.6. Sensitivity to the Threshold 33
4.2.3.7. Sensitivity to the Clustering Granularity 35
4.2.3.8. Pre-Training on Tile Labels 37
4.2.3.9. Calibration of the Scores and Ranking Utility 39
4.2.3.10. External GPS Labels: Koren et al. Table S4 41
4.2.3.11. Scope Comparison with MetaDegron 42
4.2.3.12. Experiments Not Done 43
4.2.4. Discussion 44
4.2.5. Reproducibility of the Reported Results 46
4.3. Software Testing Plan 46
4.3.1. Functions Under Test 46
4.3.2. Approach 46
4.3.3. Criteria for Test Success or Failure 47
4.3.4. Suspension Criteria and Restart Requirements 47
4.3.5. Test Deliverables 47
4.4. Test Management 47
4.4.1. Test Activities 47
4.4.2. Test Environment 47
4.4.3. Resources and Allocation 48
4.4.4. Risks 48
4.5. Test Scenarios and Test Cases 48
4.5.1. Test Scenarios 48
4.5.2. Expanded Local Acceptance Scenarios 48
4.5.3. Test Cases 49
4.6. Test Evaluation 49

PART 3. CONCLUSION 51
1. Achieved Results 51
1.1. Knowledge and Skills 51
1.2. Program and Practical Application Potential 51
2. Limitations and Challenges 51
3. Future Work 51

REFERENCES 52

APPENDIX A: INSTALLATION AND USER GUIDE 54
A.1. Installation Requirements 54
A.1.1. Hardware Requirements 54
A.1.2. Software Requirements 54
A.2. Local System Execution 54
A.2.1. Existing Baseline Commands 54
A.2.2. Target Local Workflow 55
A.3. User Guide 55

APPENDIX B: SUPPLEMENTARY MATERIALS 56
B.1. Source Code and Data Availability 56
B.2. Additional Tables and Figures 56

LIST OF FIGURES

Figure 3.1. Planned components of the single local application; arrows denote data interactions. 19

Figure 4.1. Macro AUROC against the mean number of training clusters (nested cluster subsets of 25, 50, 75 and 100% of the outer-training data; the dashed segment is the relaxed-threshold training set). . . 30

Figure 4.2. Gain in macro AUROC over the benchmark training set for each degron threshold, with 95% cluster-bootstrap intervals (horizontal offsets only separate the two models). 35

LIST OF TABLES

Table 1.1. Predict the effect matrix 6
Table 1.2. Rank substitutions 6
Table 1.3. Export results with provenance 7
Table 1.4. Select a protein and tiles 8
Table 1.5. Compare with measured data 8
Table 1.6. View ClinVar and reference annotations 9
Table 1.7. View model and data provenance 10
Table 1.8. Analyse peptide batches or a whole protein 10
Table 1.9. Import experimental observations 11
Table 1.10. Manage projects and analysis history 12

Table 3.1. Data roles in research and the first local release 20
Table 3.2. Key workspace fields and invariants 21

Table 4.1. Fate of the raw saturation peptides under the label filters 25
Table 4.2. Macro AUROC on the 91 peptides of the saturation benchmark. The last column keeps only the cells whose exact sequence is not in the training data of the published Degformer. 27
Table 4.3. Paired AUROC differences between models 28
Table 4.4. Per-seed AUROC of the three main models 28
Table 4.5. Interaction designs against ridge M3. “Criterion met” applies the pre-registered rule above; for the sigmoid gate and the one-seed Degformer architecture the rule is applied descriptively. 29
Table 4.6. Learning curve (P1) and relaxed degron threshold 4.0 (P0). Ranges are over 5 folds and 3 seeds. 31
Table 4.7. Pre-registered verdicts of P1 and P0 32
Table 4.8. Cells of the benchmark peptides and of the 85 peptides added by relaxing the threshold 32
Table 4.9. Gain over the benchmark training set when a fraction of the new clusters is added 33
Table 4.10. Pre-registered verdicts on the new clusters. The reference marginal gain of old-region clusters is 0.0038. 34

Table 4.11. Added data and gains for each degron threshold (cov80, positive cut ΔPSI ≥ 1) 34
Table 4.12. Robustness of the gain at threshold 4.0 (ridge M3) 35
Table 4.13. Gain of the relaxed threshold 4.0 (ridge M3) under different protein partitions. "Groups" are the groups of the 91 benchmark peptides and of all 176 peptides; "Shared" is the number of added peptides that fall in a group of a benchmark peptide; "Excl." is the mean percentage of the added cells that are removed from training because their group contains a test peptide. 36
Table 4.14. Macro AUROC with and without pre-training on tile labels . . . 38
Table 4.15. Pre-registered comparisons for pre-training. The main configuration is the relaxed threshold 4.0; the threshold 3.0 is reported with the same criteria and was not used to choose the configuration. . . 38
Table 4.16. Probability quality on the outer-test cells (observed fraction of positive cells: 0.131). ECE: expected calibration error over ten equal-mass bins; $\hat{p}$: predicted probability. 39
Table 4.17. Pre-registered comparisons of the Brier score 40
Table 4.18. Ranking utility: the substitutions of each peptide (532 per 28-residue peptide) are ordered by the raw score. A random ranking has enrichment 1. 41
Table 4.19. Gate fixed in advance for merging the labels of Koren et al. (Table S4) into training 41
Table 4.20. Audit of the MetaDegron data and overlap with the data of this thesis 43
Table 4.21. Planned first-release local acceptance scenarios; no execution result claimed 49
Table 4.22. Executed test cases: agreement of the prototype with the research runner. The tolerance is 10^{-9}. 50

GLOSSARY AND ABBREVIATIONS

No.	Term / Abbreviation	Definition / Explanation
1	Degron	Sequence element that marks a protein for selective degradation.
2	E3 ligase	Enzyme that recognises substrate proteins and transfers ubiquitin to them.
3	Residue	One amino acid at a specific position of a protein or peptide.
4	Footprint	Map of positions or substitutions that affect degron activity under a given assay.
5	GPS	Global Protein Stability profiling, a fluorescent-reporter assay of protein stability.
6	PSI	Protein Stability Index, a summary of the reporter signal distribution over sorting bins.
7	ΔPSI	PSI of the mutant minus PSI of the wild type.
8	WT	Wild type, the unmutated reference sequence.
9	PAP	Model that predicts degron activity of a whole peptide.
10	AUROC	Area under the receiver operating characteristic curve.
11	MMseqs2	Software for sequence search and clustering.
12	Cov80 / cov50	Protein clusters built with 80% / 50% alignment coverage.

PART 1. INTRODUCTION

1. Problem Statement

The abundance of a protein in a cell depends on both its synthesis and its degradation. In the ubiquitin–proteasome system, a *degron* is a short sequence element that marks a protein for selective degradation [1], [2]. A single amino acid substitution inside a degron can strengthen or weaken this signal, so knowing the effect of each substitution helps researchers choose mutations to test experimentally [3], [4].

Existing experiments score a whole peptide with one activity value. That value does not say which position, or which substituted amino acid, makes the degron signal stronger or weaker. Measuring every substitution experimentally is costly, which motivates a predictive model that shortlists substitutions. Such a model is only useful if it is evaluated on experimental labels and if similarity between training and test proteins is controlled.

2. History of Approaches to the Problem

2.1. International Context

Large-scale stability profiling based on the Global Protein Stability (GPS) assay [5] has produced experimental degron maps, including scanning and saturation mutagenesis of human peptides [3], cytosolic tile maps [4] and transcription-factor tile maps [6]. On the computational side, PAP and Degformer predict peptide-level degron activity or protein stability [4], [7], MetaDegron predicts E3-ligase-targeted degron motifs [8], and DEGRONOPEDIA and DegronMD provide annotation resources [9], [10].

These works address peptide- or motif-level prediction and annotation. Residue-level substitution effects measured on a common assay endpoint, evaluated with protein-cluster-aware splits, remain a question that the thesis examines rather than a gap it assumes is unfilled.

2.2. Domestic Context

[PENDING: literature search on degron and protein-stability prediction work in Vietnam; to be written only after the search is done.]

3. Objectives

3.1. General Objective

To study methods that predict the effect of amino acid substitutions in degrons from experimental data, to evaluate generalisation across protein clusters, and to build a tool that supports analysis and prioritisation of mutations.

3.2. Specific Objectives

1. Build a reproducible pipeline that normalises data, maps coordinates and derives mutation-effect labels while keeping each source and assay condition separate.

2. Establish baselines: amino acid composition, ridge regression, the PAP score difference and the Degformer architecture, under clearly stated comparable conditions.

3. Test whether representations learned from peptide-level labels and fine-tuned on mutation labels improve prediction, and study the learning curve before choosing a model.

4. Evaluate effectiveness, stability across seeds, error, and the contribution of each component.

5. Build one local React/FastAPI application for single-peptide and batch/whole-protein tile analysis, ClinVar/reference annotation, experimental-data comparison, project history and traceable export.

4. Research Subject and Scope

4.1. Research Subject

Degron peptides, single amino acid substitutions, protein stability indices measured with a reporter assay, and models predicting residue-level mutation effects. A *residue* is one amino acid at a specific position; the *footprint* is the map of positions or substitutions that change degron activity under a given assay.

4.2. Research Scope

- The main label source is the human peptide data of Zhang et al. [3]; other peptide-level sources are used only for transfer learning or domain studies when their terms of use allow.
- Input: reference (wild-type) sequence, position and substituted amino acid. Output: predicted effect ΔPSI, which can be arranged into an $L \times 20$ matrix.
- The first software release includes peptide batches, whole-protein tiling, ClinVar and approved reference annotations, CSV/TSV experimental import for comparison, and persistent local projects and analysis history. Each scored tile remains a 28-residue peptide; whole-protein processing does not validate intact-protein effects. Imported observations do not automatically enter training.
- One local application is required; public hosting, public sharing, account management and remote deployment are outside acceptance. Saved project work persists until user deletion.
- Identifying the specific E3 ligase is not a main task. No new wet-lab experiments are performed, and software output supports hypothesis generation; it does not establish causation, disease effects or therapeutic benefit.

5. Research Process and Technologies Used

5.1. Research Process

The work combines a literature review, controlled computational experiments and a software prototype: (1) study degron biology and related methods; (2) audit data and design the evaluation; (3) train and compare models; (4) develop and test the prototype; (5) analyse results and write the thesis.

5.2. Technologies Used

Python with NumPy, pandas, scikit-learn and PyTorch for modelling, and MMseqs2 for protein clustering [11]. The application uses React with TypeScript (Vite), FastAPI and local PostgreSQL, with a local worker for durable CPU inference jobs; SQLite supports isolated tests and baseline development.

5.3. Development and Supporting Tools

Git for version control and LaTeX (XeLaTeX) for this report. The local software test plan uses pytest, API/schema checks and browser tests. Docker may package the same local components; a public-host deployment is not required.

6. Main Contributions

[PENDING: to be written from achieved results at the end of the work. Planned contributions are a reproducible data and evaluation pipeline, a documented comparison with baselines including negative results, and a prototype tool.]

7. Structure of the Thesis

Part 1 introduces the problem, objectives and scope. Part 2 has four chapters: Chapter 1 describes the problem and the system requirements; Chapter 2 gives the theoretical background; Chapter 3 presents the design of the data, method and system; Chapter 4 covers experimental evaluation and software testing. Part 3 states the results, limitations and future work. The appendices give installation and usage instructions and supplementary material.

PART 2. CONTENT

CHAPTER 1: PROBLEM DESCRIPTION

1.1. Detailed Problem Description

Researchers selecting degron mutations for follow-up experiments face a limited experimental budget. A 28-residue peptide has 532 possible single-amino-acid substitutions. The software must help the researcher inspect these candidates, choose a shortlist and retain the evidence needed to revisit that choice. Its primary deliverable is a traceable prioritisation workflow.

The thesis connects two problems. The research pipeline predicts the effect ΔPSI of a substitution from a reference peptide, position and replacement amino acid, using experimental labels and protein-cluster-aware evaluation. The application uses a frozen model to support analysis, annotation and comparison. The model task remains mutation-level prediction; annotation and project management do not change its scientific endpoint.

First-release scope. The software scope confirmed on 29 October 2026 is one local web application using React with TypeScript and FastAPI. It includes single-peptide analysis, peptide batches, whole-protein tiling, ClinVar and approved reference annotations, import of experimental observations for comparison, CSV/JSON export, and persistent projects and analysis history. The browser, API, local worker and database belong to the same local workspace. Public hosting, remote deployment and public sharing are outside acceptance; no separate public and internal editions are required.

The main actor is the researcher operating this local workspace. Account management, multi-user collaboration, wet-lab/sample management, automatic retraining and clinical diagnosis are outside scope. Imported measurements remain project data and are not added automatically to training or benchmark labels. No new biological experiment is required by this thesis.

1.2. Functions of the System

The following use cases describe the target first release, not completed implementation. Baseline single-tile API/frontend code and a CLI already exist; the added

project, job and import flows still require implementation and verification. The requirements below define the target behaviour; Chapter 4 specifies the corresponding acceptance scenarios.

1.2.1. Predict the Effect Matrix

Table 1.1. Predict the effect matrix

Use case name: Predict effect matrix	**ID:** UC-01
Primary actor: Researcher	**Necessity level:** Important **Classification:** Simple
Stakeholders and interests: Researcher: inspect the effect of every single substitution of a peptide.	
Summary description: Enter a 28-residue WT peptide or select a reference tile; obtain 532 scores in a position-by-amino-acid grid with WT cells masked.	
Relationships: Include: validate input and assess domain. Extend: compare observations, view annotations, save analysis.	
Event handling: 1. Enter/select the peptide. 2. Validate syntax, coordinates and domain. 3. Score with a frozen model. 4. Show domain and warnings before the matrix.	
Alternative / exceptional flows: Invalid input: no scores. Known out-of-domain tile: refuse by default; explicit override records a warning. Unknown, unreliable or borderline domain: score only with a prominent limitation label.	
Expected outcome: A transient preview with raw ΔPSI score, calibrated $P(\Delta\text{PSI} \geq 1)$, model identity and domain evidence; explicit save invokes UC-10.	

1.2.2. Rank Substitutions

Table 1.2. Rank substitutions

Use case name: Rank substitutions	**ID:** UC-02
Primary actor: Researcher	**Necessity level:** Important **Classification:** Simple

Stakeholders and interests: Researcher: choose a shortlist within an experimental budget.
Summary description: Sort/filter substitutions within each tile and select candidates.
Relationships: Include: UC-01 or a completed tile of UC-08.
Event handling: 1. Open ranking. 2. Sort by effect/probability and filter by position or replacement. 3. Select candidates; inspect the top-10% flag.
Alternative / exceptional flows: No result: request an analysis first. Empty filter: show an explicit empty state.
Expected outcome: A tile-specific shortlist retaining coordinates and warnings; no implied cross-protein biological ranking.

1.2.3. Export Results with Provenance

Table 1.3. Export results with provenance

Use case name: Export results	**ID:** UC-03
Primary actor: Researcher	**Necessity level:** Important **Classification:** Simple
Stakeholders and interests: Researcher: reuse results in an experimental or computational workflow.	
Summary description: Export a single analysis or complete job as CSV/JSON, including selected annotations/comparisons.	
Relationships: Uses: UC-01, UC-05, UC-06, UC-08 and UC-10.	
Event handling: 1. Choose scope and format. 2. Write scores, per-item outcomes, coordinates, model/data versions and warnings.	
Alternative / exceptional flows: Export failure preserves the result for retry. Failed/skipped job items remain in the manifest.	

Expected outcome: Complete traceable files, including rows beyond the current page; no public share link.

1.2.4. Select a Protein and Tiles

Table 1.4. Select a protein and tiles

Use case name: Select protein and tiles	**ID:** UC-04
Primary actor: Researcher	**Necessity level:** Important **Classification:** Simple
Stakeholders and interests: Researcher: locate a known protein region without manually copying coordinates.	
Summary description: Search the loaded human UniProt release; choose a tile or analyse the complete protein through tiling.	
Relationships: Continues to UC-01 or UC-08.	
Event handling: 1. Search accession/gene. 2. Select the reference sequence. 3. Inspect tiles, domain states and coverage. 4. Choose the analysis scope.	
Alternative / exceptional flows: No accession, sequence shorter than 28, non-standard residues or unverified mapping: state the reason and affected coverage.	
Expected outcome: Selected reference release and explicit 1-based peptide/protein coordinates.	

1.2.5. Compare with Measured Data

Table 1.5. Compare with measured data

Use case name: Compare observations	**ID:** UC-05
Primary actor: Researcher	**Necessity level:** Important **Classification:** Simple
Stakeholders and interests: Researcher: inspect where prediction agrees or disagrees with measurements.	

Summary description: Choose reference observations or a committed project dataset; align exact WT, position and replacement.
Relationships: Uses: UC-01/08 and UC-09.
Event handling: 1. Select dataset and replicate/aggregation. 2. Check mapping, endpoint and units. 3. Display matched values, exclusions and applicable residuals/metrics.
Alternative / exceptional flows: Missing/unmatched values are not zero. Different assays remain separate without pooled ΔPSI metrics. Insufficient/constant pairs have explicit undefined-metric states.
Expected outcome: Comparable prediction/observation views with sample counts and provenance; training observations labelled and imports not automatically called independent validation.

1.2.6. View ClinVar and Reference Annotations

Table 1.6. View ClinVar and reference annotations

Use case name: View annotations	**ID:** UC-06
Primary actor: Researcher	**Necessity level:** Important **Classification:** Simple
Stakeholders and interests: Researcher: relate candidate substitutions to published variant and reference evidence.	
Summary description: Show ClinVar missense records, UniProt mapping and approved GPS evidence in release 1.	
Relationships: Extends: UC-01/08; provenance from UC-07.	
Event handling: 1. Enable a layer. 2. Verify accession/release/WT residue. 3. Show classification, review/conflict state, source version, attribution and disclaimer.	

Alternative / exceptional flows: Distinguish no match, unmapped custom input, unavailable source and conflicting mapping. Do not guess canonical mappings.
Expected outcome: Traceable annotations separate from effect predictions; annotation absence does not mean benign.

1.2.7. View Model and Data Provenance

Table 1.7. View model and data provenance

Use case name: View model and sources	**ID:** UC-07
Primary actor: Researcher	**Necessity level:** Important **Classification:** Simple
Stakeholders and interests: Researcher and examiner: understand the predicted event, evidence and limitations.	
Summary description: Read frozen model identity, domain, development metrics and source licences/versions.	
Relationships: Supplies evidence to all result/export views.	
Event handling: 1. Open model/data information. 2. Inspect current metadata or the snapshot associated with a saved run.	
Alternative / exceptional flows: Missing historical artefact: report an integrity problem instead of silently substituting a newer source.	
Expected outcome: Explicit distinction between development evidence, imported comparison and independent validation.	

1.2.8. Analyse Peptide Batches or a Whole Protein

Table 1.8. Analyse peptide batches or a whole protein

Use case name: Batch/protein analysis	**ID:** UC-08

Primary actor: Researcher	**Necessity level:** Important **Classification:** Complex
Stakeholders and interests: Researcher: analyse many candidates while retaining item-level failures and context.	
Summary description: Submit bounded peptide records/FASTA or one protein accession/FASTA to a persistent local job.	
Relationships: Uses UC-01/04/10; exports through UC-03.	
Event handling: 1. Choose a project and input mode. 2. Preview validation, tiling and coverage. 3. Confirm local persistence and submit. 4. Track progress and open per-tile results.	
Alternative / exceptional flows: Item failures/refusals preserve successes. Cancellation/restart retain completed results without duplicates. Limits and uncovered residues are explicit.	
Expected outcome: A durable job with reconciled outcome counts, pinned model/source versions and distinct overlapping tile contexts.	

1.2.9. Import Experimental Observations

Table 1.9. Import experimental observations

Use case name: Import observations	**ID:** UC-09
Primary actor: Researcher	**Necessity level:** Important **Classification:** Complex
Stakeholders and interests: Researcher: compare predictions with locally supplied measurements.	
Summary description: Validate CSV/TSV plus assay, units, source, replicate and reference metadata; commit a dataset revision.	
Relationships: Uses UC-10; supplies UC-05.	

Event handling: 1. Upload into a project. 2. Preview row-level errors, duplicates, missingness and mapping. 3. Correct blocking errors and acknowledge warnings. 4. Explicitly commit.
Alternative / exceptional flows: Malformed/nonfinite/WT-conflicting records block commit; conflicting duplicates never overwrite. Other endpoints remain separately labelled.
Expected outcome: An immutable project dataset with file hash and provenance; no update to training data or weights.

1.2.10. Manage Projects and Analysis History

Table 1.10. Manage projects and analysis history

Use case name: Manage projects/history	**ID:** UC-10
Primary actor: Researcher	**Necessity level:** Important **Classification:** Complex
Stakeholders and interests: Researcher: revisit the exact analysis used for an earlier experimental decision.	
Summary description: Create/rename/reopen/delete local projects; save and reopen immutable runs and datasets.	
Relationships: Supports UC-01/03/05/08/09.	
Event handling: 1. Create/select project. 2. Explicitly save a preview or submit/commit work. 3. Reopen history after restart. 4. Rerun as a new entry or confirm deletion after a dependency summary.	
Alternative / exceptional flows: Missing snapshot or failed cleanup is explicit; deletion cannot remove shared reference/model data or other projects.	
Expected outcome: Saved input, configuration, results and provenance retained locally until deletion; no account or public-sharing requirement.	

1.3. Non-Functional Requirements

The detailed identifiers and cases are in the system SRS. The targets below are proposed until measured on the recorded local computer; no external-host performance or availability result is required.

1.3.1. Performance Requirements

Proposed p95 targets are 300 ms for one tile, 200 ms for listing a 1,000-residue protein's tiles and 1.5 s from click to rendered heatmap. Batch/import resource use is measured separately; bounded jobs, pagination and cancellation keep the interface usable. Proposed initial bounds are 100 peptides or one protein of at most 10,000 residues per job, and 10 MiB/100,000 rows per import; these are configurable design defaults, not measured capacities.

1.3.2. Information Safety Requirements

Serve only approved reference data with source versions and required attribution. ClinVar retains its review/conflict information and diagnostic disclaimer. Imported observations stay in a project-specific store with assay/units/replicates and are not automatically pooled into reference or training data.

1.3.3. Security Requirements

Bind application services to loopback and restrict browser origins. Validate all input and uploaded files; never execute their contents or use supplied names as paths. Unsaved previews are transient. Explicit saves, job submissions and import commits persist locally until deletion; no sequences or observations appear in logs, URLs or third-party telemetry. Project history has no automatic 30-day expiry.

1.3.4. Reliability Requirements

Score arrays are deterministic for the same normalised input and frozen configuration. Numerical agreement with the runner must be below 10^{-9} for JSON and 5×10^{-9} for ten-digit CSV. Saved snapshots survive restart; job recovery does not duplicate completed results. Missing data, partial failures and invalid mappings remain visible.

1.3.5. Maintainability Requirements

Version models, source snapshots and schema migrations. Document local API/worker/frontend startup and workspace recovery. Test the clean local workflow rather than cloud installation.

1.3.6. Usability and Accessibility Requirements

Provide keyboard workflows, accessible tables and text/patterns in addition to colour, with WCAG 2.1 AA as the accessibility target. Desktop use is primary; smaller screens use a scrollable matrix/table fallback.

1.4. Implementation and Design Constraints

1.4.1. Implementation Constraints

One student implements the system within the thesis schedule. React with TypeScript and FastAPI are fixed choices; PostgreSQL is local project/reference storage and SQLite supports isolated tests/baseline development. A local worker runs CPU inference. Training and research notebooks remain separate. Docker may package the same local application; public deployment is not a deliverable.

1.4.2. Design Constraints

Whole-protein analysis retains full 28-residue windows at step 23 and reports uncovered positions. It does not establish the behaviour of the intact protein. Every result carries assay/domain evidence; raw ΔPSI, calibrated event probability and clinical annotation are separate quantities. The scope update does not establish clinical validity, an E3 mechanism or completion of research gates.

CHAPTER 2: THEORETICAL BACKGROUND

2.1. Domain Background

2.1.1. Proteins, Amino Acids and Residues

A protein is a chain of amino acids. A *residue* is one amino acid at a specific position of the chain. A *single amino acid substitution* replaces the residue at one position with another of the 20 standard amino acids.

2.1.2. The Ubiquitin–Proteasome System and Degrons

In the ubiquitin–proteasome system, E3 ubiquitin ligases recognise substrate proteins and attach ubiquitin chains to them; ubiquitinated proteins are then degraded by the proteasome. A *degron* is a sequence element that is part of this recognition and marks a protein for selective degradation [1], [2]. Degrons can lie at the protein termini or inside the sequence [3]. The activity of a peptide taken out of its protein may differ from its activity in the intact protein, because surface exposure, structure and cellular context matter.

2.1.3. Measuring Degron Activity

Global Protein Stability (GPS) profiling [5] fuses a peptide to a fluorescent reporter and sorts cells by fluorescence into bins. The *Protein Stability Index* (PSI) summarises the distribution of cells over the bins; in the data of Zhang et al. [3], a lower PSI corresponds to a less stable reporter. The effect of a substitution is defined as

$$\Delta\mathrm{PSI} = \mathrm{PSI}_{\mathrm{mutant}} - \mathrm{PSI}_{\mathrm{WT}}, \tag{2.1}$$

where a positive value means the reporter is more stable under the assay. This describes the measured reporter stability and does not by itself show direct recognition by a specific E3 ligase.

Scanning mutagenesis replaces residues in a systematic way along the peptide, whereas *saturation* mutagenesis tests many amino acids at each position. Cells from the same peptide are statistically related and cannot be treated as fully independent samples. Measurement noise comes from read counts, replicate variation and experimental conditions.

2.2. Methods and Models

2.2.1. Problem Formulation

The task is to predict ΔPSI for a (peptide, position, substituted amino acid) triple. Predictions for all positions and amino acids form a matrix of size $L \times 20$ for a peptide of length L. For evaluation, ΔPSI can also be thresholded into a binary label; the threshold and the direction of the effect must be fixed in the evaluation protocol before models are compared.

2.2.2. Baseline and Reference Models

- **Ridge regression** [12]: a linear model with L_2 regularisation, used as a simple baseline on encodings of the reference and substituted amino acids, peptide composition and local context.

- **PAP score difference**: PAP predicts a degron score for a whole peptide [4]; the mutation effect is taken as $\mathrm{PAP}_{\mathrm{mutant}} - \mathrm{PAP}_{\mathrm{WT}}$.

- **Degformer**: a transformer-based model [13] that predicts PSI [7].

MetaDegron [8] predicts degron motifs and is used to compare problem scope, not as a label source.

2.2.3. Transfer Learning

Transfer learning pre-trains a representation on peptide-level activity labels and then fine-tunes it on mutation-level labels. The two label types have different meanings, so they need separate prediction heads or a suitable transformation; binary motif labels are not converted into ΔPSI values.

2.3. Technologies and Tools

Python with NumPy, pandas and scikit-learn for data handling and classical models; PyTorch for neural models; MMseqs2 [11] for clustering protein sequences. Versions are recorded when experiments are run. The local application uses React with TypeScript, FastAPI and PostgreSQL, with a local worker for batch inference. This runtime design is separate from the research toolchain.

2.4. Evaluation Methodology

2.4.1. Splitting by Protein Cluster

Randomly splitting individual cells lets similar sequences appear in both training and test sets and inflates results. Proteins are therefore clustered by sequence similarity (MMseqs2, 30% identity, 80% coverage as the main setting; 50% coverage as a sensitivity check) and whole clusters are assigned to folds. Hyperparameters are chosen on a cluster-wise validation set inside each training fold; scaling, feature selection and pre-training must not use the held-out clusters.

2.4.2. Metrics

The main metric is the cell-level area under the ROC curve (AUROC), computed per peptide that contains both classes and then averaged. Spearman correlation or mean absolute error on continuous ΔPSI is added when labels allow.

2.4.3. Statistical Comparison

Models are compared pairwise on the same peptides. Confidence intervals come from bootstrap resampling [14] by protein cluster, and results are reported per random seed in addition to the aggregate. A development benchmark is distinguished from an independent validation.

CHAPTER 3: SOLUTION DESIGN AND IMPLEMENTATION

This chapter describes the target design confirmed on 29 October 2026. Baseline API/frontend code exists, but the workspace, durable jobs and import extensions below are planned. A design element is not a completed implementation or a passing test. The design follows the requirements in Chapter 1; existing implementation evidence is distinguished from planned work.

3.1. System Overview

3.1.1. Product Context

The research pipeline prepares data, trains and evaluates models offline. One local web application loads a frozen model and reference snapshots; it supports researchers choosing substitutions for follow-up experiments. Runtime input and project observations stay on the local computer.

3.1.2. Functional Overview

The first release covers UC-01 to UC-10 of Chapter 1: prediction, ranking, export, protein/tile lookup, experimental comparison, ClinVar/reference annotations, evidence pages, peptide/protein jobs, experimental import and persistent project history. There is one product configuration with local operation; public hosting and public sharing are excluded.

3.2. System Architecture

3.2.1. Architectural Components

Figure 3.1 shows the target components. All application listeners use loopback; reference downloads and training are preparation activities outside the runtime workflow.

The React client owns input, matrix/ranking interaction and project/job/import screens. FastAPI validates input, applies domain rules, serves reference annotations and stores snapshots. A local worker uses the same inference package for jobs; PostgreSQL stores job state and project-owned records. Generated local file paths hold committed imports and versioned artefacts. Reference ETL never writes user observations into research data.

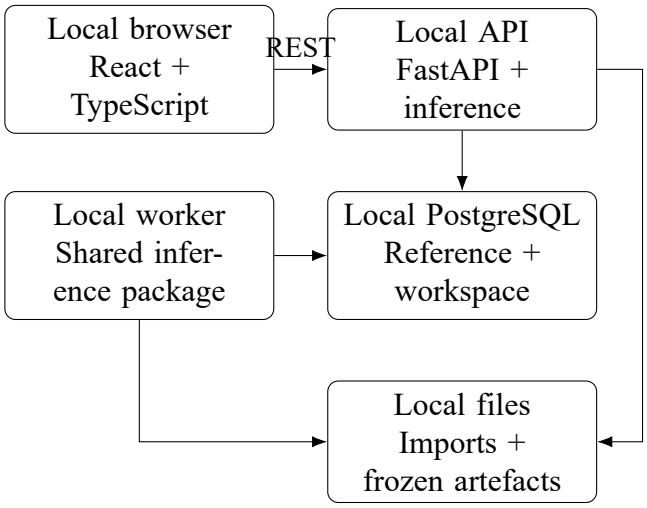


Figure 3.1. Planned components of the single local application; arrows denote data interactions.

3.2.2. Interaction Between Components

A single-tile preview is validated and scored without persistence. Explicit Save creates an immutable analysis in a selected project. Batch submission pins model, calibrator, gate and source versions, returns a job ID, then the worker claims items transactionally. The client polls progress and opens per-tile results. Expired worker leases are recovered after restart; completed items are not duplicated. Cancellation and project deletion prevent late writes.

An import first produces a validation preview. Only a valid, explicitly committed dataset becomes available for comparison. A comparison selects a saved analysis and dataset revision, checks assay and mutation alignment, and stores its own result/provenance. Reading history uses saved snapshots, not a new inference call or the latest ClinVar release.

3.2.3. Design Rationale

React supports the interactive matrix, filtered tables and long-running-task states. FastAPI keeps the model computation in Python and provides an explicit REST contract. A local worker separates job execution from interactive requests without introducing a cloud service. Persistent projects require a database and snapshot lifecycle beyond the baseline transient preview. The trade-offs are local resource limits, file/database consistency and additional recovery/deletion tests. SQLite remains useful for isolated tests, while local PostgreSQL integration is an acceptance task. The

OpenAPI contract marks new operations as planned; generating a schema does not implement them.

3.3. Data Design

3.3.1. Data Sources and Preprocessing

Reference snapshots have version/date, checksum and terms of use. Research-only sources and project-imported observations are kept distinct from served reference data (Table 3.1).

Table 3.1. Data roles in research and the first local release

Source	Role and boundary
Zhang Mendeley v2 [15]	Mutation labels and reference observations; training use labelled.
GEO GSE240610 [16]	GPS screen context; preserve estimator and source provenance.
UniProtKB/Swiss-Prot	Pinned human reference sequences, accessions and coordinate/WT checks.
ClinVar	First-release missense annotation; source/review/conflict state, attribution and diagnostic disclaimer.
Project CSV/TSV	User observations for assay-aware comparison only; preserve source, units and replicates.
Voutsinos/PAP [4]	Research baselines and transfer-learning studies; not a replacement for mutation labels.
Larsen [6] and other restricted sources	Research-only consideration subject to rights; not automatically imported into the service.

3.3.2. Data Model

Reference entities are source, protein, tile, measured cell, ClinVar variant and model version. The planned workspace adds project, analysis job, job item, analysis run/cell, imported dataset/observation and comparison. Legacy expiring share tables are not reused as persistent project history.

3.3.3. Data Dictionary

3.3.4. Relationships Between Entities

A project owns jobs, runs and datasets. A job contains input items; each successful tile item produces one immutable run with 532 cells. Dataset observations retain unique mutation/replicate keys. A comparison joins one run and one com-

Table 3.2. Key workspace fields and invariants

Entity	Required information
Project	UUID, name, description, creation/update time; one local workspace.
Job and item	Mode/input IDs, pinned configuration, states/counts, item errors, coverage and worker lease; unique completed result per item.
Analysis run/cell	Input and coordinate context; model/gate hashes; scores, rank, warnings, annotation/source snapshot and parent rerun ID.
Dataset/observation	File hash/revision, assay/endpoint/units/source, WT sequence, position, WT/replacement, value/missing reason and replicate.
Comparison	Run/dataset IDs, explicit replicate/aggregation, compatibility decision, alignment counts and metric/result snapshot.

mitted dataset from the same project. Source/model artefacts are versioned independently; changing active reference data must not change an old analysis. Deletion shows dependencies, stops related work and removes owned files/results with retryable cleanup; shared references and weights remain intact. Committed history has no automatic 30-day expiry.

3.4. Method and Model Design

Input remains the reference peptide, position and replacement; target is ΔPSI. The existing service baseline is the frozen ridge/Platt model from Section 4.2.3.9. This scope update does not choose a replacement model or change the research comparisons with PAP/Degformer. Raw effect and calibrated $P(\Delta\text{PSI} \geq 1)$ remain separate fields.

Domain integration. The effect model's intrinsic-WT training threshold must not be applied directly to the GPS screen estimator. The R3 research provides a two-tier gate with an intermediate borderline state; integrating its frozen rules and provenance into the baseline remains pending. Every tile has domain evidence; known out-of-domain scoring requires an explicit warned override. Whole-protein mode is a collection of peptide-context predictions, not a validated intact-protein model.

3.5. Functional Design

3.5.1. Batch and Protein Processing

Peptide mode accepts a bounded list or multi-record FASTA of 28-residue inputs. Protein mode accepts one reference accession or one FASTA record. Tile n starts at $(n-1) \cdot 23 + 1$ and ends 27 residues later; only full windows are generated. Report uncovered intervals, invalid windows and domain refusals. Overlapping protein substitutions retain their tile contexts without averaging. Job states are pending, running, completed, partial, failed or cancelled; per-item states and counts explain partial results. Browser tables are paginated; exports include every outcome.

3.5.2. Import and Comparison

CSV/TSV rows contain peptide ID, WT sequence, 1-based position, WT/replacement, observed value and replicate. Optional accession/release/start coordinates are checked together. Metadata identify assay, endpoint, units, source and reference. Duplicate/conflicting mutation-replicate keys, nonfinite values and WT mismatches block commit. Empty observations retain a missing reason.

Comparison requires exact mutation alignment and explicit replicate selection or aggregation. Comparable GPS ΔPSI supports residuals (prediction minus observation), MAE and Spearman with sample counts; insufficient/constant pairs have no reported correlation. Different assay endpoints remain separate without pooled metrics. Imported observations do not automatically constitute independent validation or become training labels.

3.5.3. Annotations and Project Screens

The first release shows ClinVar alongside approved UniProt/GPS context. Show classification, review/conflict state, source release and WT-coordinate agreement; distinguish no match from unavailable/ambiguous mapping. Attribution and diagnostic disclaimer travel with saved/exported annotations. Custom unmapped sequences have explicit annotation-unavailable states. The project workspace exposes history, jobs and datasets; users can reopen snapshots, create reruns as new entries and confirm deletions. New screens and API routes remain pending implementation.

3.6. Local Execution View

One computer hosts the browser, API, worker, PostgreSQL and workspace files. Application listeners bind to loopback; the frontend origin is explicitly allowed. Docker may package the same components, but no remote host, public URL, TLS deployment or separate product edition is required. Appendix A separates baseline startup from the pending first-release workflow.

CHAPTER 4: TESTING AND EVALUATION

4.1. Testing Objectives

To check that the research pipeline gives reproducible results without leakage between training and test clusters, and that the prototype validates input, keeps coordinates and score direction correct, and exports data consistently with the research runner.

This chapter has two parts. Section 4.2 reports the experimental evaluation of the research pipeline; these experiments have been run. Sections 4.3 to 4.6 follow the test-plan outline of the Faculty for the prototype tool. The numerical results retained here come from the recorded CLI verification artefacts. Baseline web/API code and historical local checks also exist, but the first-release extensions confirmed on 29 October 2026 require implementation and new local acceptance evidence; no passing result for those additions is claimed here.

4.2. Experimental Evaluation

4.2.1. Experimental Setup

Data and labels. The labels come from the saturation-mutagenesis experiment of Zhang et al. [3], [15]. For each peptide, the effect of a substitution is $\Delta\text{PSI} = \text{PSI}_{\text{mutant}} - \text{PSI}_{\text{WT}}$ (Chapter 2). The data set contains two replicate halves of each peptide. The PSI_{WT} reference of a half is the median PSI of the synonymous cells of that half, and at least ten such cells are required. The cell label is the mean ΔPSI of the two halves; a cell is *positive* when $\Delta\text{PSI} \geq 1$. A peptide is accepted as a degron only if $\text{PSI}_{\text{WT}} \leq 3.0$. Table 4.1 shows how the 248 raw saturation peptides reduce to the 91 peptides of the benchmark. The single largest cause of loss is the degron threshold, which removes 144 peptides; the technical causes together remove only $10 + 2 + 1 = 13$ peptides. The benchmark set contains 91 peptides in 89 protein clusters and 47 847 cells. The measurement ceiling of this set, computed as the agreement between its two experimental halves, is 0.9519 AUROC.

Splits. Proteins are clustered with MMseqs2 [11] at 30% identity and 80% coverage (cov80). The outer evaluation uses five folds with whole clusters assigned to folds.

Table 4.1. Fate of the raw saturation peptides under the label filters

Outcome of the label filters	Peptides
Retained: the benchmark set	91
Removed only by the degron threshold ($PSI_{WT} > 3.0$)	144
Fewer than 10 synonymous cells for the WT reference in a half	10
Sequence mismatch between scan file and UniProt	2
No sequence could be derived	1
Raw saturation peptides	*248*

Inside each outer training set, 20% of the clusters form a validation set for choosing hyper-parameters, so no outer-test label is ever used for tuning. Training is repeated with seeds 17, 29 and 43. The three seeds share the same outer partition and differ only in the inner validation split and in network initialisation, so they are not three independent partitions. The reported score of a model is the AUROC of the average of the three seeds' out-of-fold scores; per-seed results are also kept.

Models.

- **Ridge M3.** Ridge regression [12] on 180 features: one-hot reference and substituted amino acid (40), peptide composition (20) and the ordered ± 3 residue context (120). The penalty is chosen from $\{1, 10, 100, 1000\}$ on the inner validation set.
- **ΔPAP.** The difference of the PAP degron score [4] between mutant and wild type, with the published network frozen (variant `cnn2w1`). It is the reference baseline and uses no training labels.
- **Gated networks.** The same 180 features, with the substitution (pair) and the composition-plus-context inputs projected separately to 32 dimensions; a sigmoid of the pair projection multiplies the context representation, followed by an MLP ($64 \to 32 \to 1$). Variants add the frozen ΔPAP as a residual or remove the gate. Training uses AdamW (learning rate 2×10^{-3}, weight decay 0.01, batch of about 1,024, at most 60 epochs, patience 8).
- **Degformer.** The published checkpoint (epoch 50) of the transformer [7], scored as $\text{pred}(\text{mutant}) - \text{pred}(\text{WT})$; and the same architecture (d_model 128, 4 layers,

8 heads) retrained from scratch on the training clusters of each fold.

Further interaction designs (FiLM, rank-8 bilinear, ridge with cross-product features) were added as *pre-registered candidates* in Section 4.2.3.3.

Pre-registration. The acceptance criteria of the overlap check, the interaction designs, the retrained Degformer architecture, the learning-curve and threshold experiments, the re-clustering, the pre-training experiment, the calibration and the audit of the external labels were written to a protocol file before the experiments were run, and were not changed afterwards. Where a result below is described as met or not met, it refers to those fixed criteria. Observations made after seeing the numbers are labelled as descriptive. The first direct-learning experiment (Table 4.2) has a protocol file but is treated as exploratory.

Environment. All experiments ran on CPU with Python 3.12.3, NumPy 2.5.3 and PyTorch 2.14.0 (CPU build); each result file records the SHA-256 of its input and of its runner. Large caches and checkpoints are generated again by the runners and are not kept in the repository.

4.2.2. Evaluation Metrics

The main metric is the cell-level AUROC (positive: $\Delta\text{PSI} \geq 1$), computed per peptide that has both classes and then averaged over peptides (macro AUROC). Paired differences between two models are computed on the same peptides. The 95% confidence intervals come from 5,000 bootstrap resamples of protein clusters [14]; every peptide of a sampled cluster is kept, so cells and peptides are not treated as independent. These intervals are conditional on the stored out-of-fold predictions and do not include the variance of retraining. No correction is made for the many comparisons, so a single interval is a development-stage indication and not a confirmation.

Probabilities are assessed with the Brier score (the mean squared difference between the predicted probability and the 0/1 label over the cells), the Brier skill score against a constant forecast of the base rate, $\text{BSS} = 1 - \text{Brier}/\text{Brier}_{\text{const}}$, and the expected calibration error (ECE) over ten equal-mass bins, pooled over the outer-test cells and bootstrapped by cluster in the same way. Ranking utility is the *enrichment at* $k\%$: for each peptide, the substitutions are ordered by predicted score, the precision

of the top $k\%$ is divided by the fraction of positive cells of that peptide, and the ratio is averaged over the peptides that have both classes; a random ranking has enrichment 1.

4.2.3. Results

All results below are on a *development benchmark*: the same 91 peptides have been examined in several experiments, and the out-of-fold design is not an independent validation on unseen data.

4.2.3.1. Direct Learning on the Saturation Matrix

Table 4.2 lists the macro AUROC of each model. Ridge M3 reaches 0.8082, above the frozen ΔPAP baseline (0.7693). The paired difference is +0.0389 with a cluster-bootstrap interval [+0.0216, +0.0571] (Table 4.3). The gated network with residual ΔPAP reaches 0.8002 and is below ridge (difference −0.0080, [−0.0152, −0.0006]). Removing the gate (0.7937) or the ΔPAP residual (0.8015) does not change this picture: both variants stay below ridge.

Table 4.2. Macro AUROC on the 91 peptides of the saturation benchmark. The last column keeps only the cells whose exact sequence is not in the training data of the published Degformer.

Model	All cells	Cells without exact-sequence overlap
ΔPAP, frozen (reference)	0.7693	0.7531
Ridge regression on M3 features	0.8082	0.8040
Gated network + residual ΔPAP	0.8002	0.7956
Network without gate, with ΔPAP	0.7937	0.7892
Gated network without ΔPAP	0.8015	0.7940
Degformer, published checkpoint (overlaps its training data)	0.9248	0.9227

The result of the gate is not stable across seeds (Table 4.4). In seed 29 the gated network is *below* the network without the gate (difference −0.0076, [−0.0152, −0.0005]), whereas it is above in the other two seeds. The ensemble figure should therefore not be read as evidence that the gate helps.

Table 4.3. Paired AUROC differences between models

Paired comparison (same peptides)	Difference in AUROC [95% cluster-bootstrap CI]
Ridge − ΔPAP	+0.0389 [+0.0216, +0.0571]
Gated + ΔPAP − ΔPAP	+0.0309 [+0.0125, +0.0501]
Gated + ΔPAP − ridge	−0.0080 [−0.0152, −0.0006]
Gated + ΔPAP − no gate	+0.0065 [+0.0027, +0.0102]
Gated + ΔPAP − gated without ΔPAP	−0.0013 [−0.0048, +0.0023]

Table 4.4. Per-seed AUROC of the three main models

Seed	Ridge	Gated +ΔPAP	No gate	Gated − no gate [95% CI]
17	0.8062	0.7895	0.7806	+0.0089 [+0.0038, +0.0145]
29	0.8068	0.7762	0.7838	−0.0076 [−0.0152, −0.0005]
43	0.8081	0.7881	0.7816	+0.0064 [+0.0013, +0.0113]

4.2.3.2. Published Degformer Checkpoint and Retrained Architecture

The published Degformer checkpoint scores 0.9248, but this is not a held-out result: 42 163 of the 47 847 mutant cells (88.12%) have an exact sequence in its training file, and the wild type of all 91 peptides is in it as well. A natural suspicion is that this exact overlap alone explains the high score. Restricting the evaluation to the 5 684 cells without overlap leaves the score essentially unchanged (0.9227, Table 4.2), so exact cell overlap does not explain the gap. The remaining leak is at the level of the peptide: the model has seen most of the matrix of every peptide, and removing a few dozen cells does not create an unseen peptide. The comparison of the published checkpoint with the models of this thesis is therefore not a fair test of generalisation, and no claim is made about it.

A fair comparison of the architecture requires the same label budget and the same cluster-wise split. Retrained under these conditions (one seed, hyper-parameters as published by its authors, target the absolute PSI), the Degformer architecture reaches 0.7500, below ridge (difference −0.0582, [−0.0734, −0.0426]) and below ΔPAP (Table 4.5). This lower score is not necessarily a property of the architecture itself: it may reflect the small number of training clusters (see Section 4.2.3.4).

4.2.3.3. Interaction Designs

Three interaction designs were fixed before running: FiLM, a rank-8 bilinear form and ridge with 2,400 cross-product features. A candidate was to be called a useful architectural component only if the following held at the same time: (a) its paired difference to ridge is positive in all three seeds, (b) the confidence interval of the ensemble difference excludes zero, and (c) the same holds against ΔPAP. None of the candidates met the criterion (Table 4.5). The best of them, FiLM, is −0.0054 ([−0.0136, +0.0028]) relative to ridge and is below ridge in every seed. The sigmoid gate of the first experiment gives the same direction, and the retrained Degformer architecture is far lower. All candidates are above the frozen ΔPAP, but so is plain ridge.

Table 4.5. Interaction designs against ridge M3. "Criterion met" applies the pre-registered rule above; for the sigmoid gate and the one-seed Degformer architecture the rule is applied descriptively.

Candidate	AUROC	Candidate − ridge [95% CI]	Per-seed difference	Criterion met
Sigmoid gate +ΔPAP (G2 runner)	0.8002	−0.0080 [−0.0152, −0.0006]	−0.0168, −0.0306, −0.0200	No
FiLM (pair-generated scale and shift)	0.8028	−0.0054 [−0.0136, +0.0028]	−0.0104, −0.0162, −0.0091	No
Rank-8 bilinear	0.7933	−0.0149 [−0.0227, −0.0072]	−0.0212, −0.0275, −0.0335	No
Ridge with 2,400 cross-product features	0.7924	−0.0158 [−0.0227, −0.0086]	deterministic	No
Degformer architecture, retrained (one seed)	0.7500	−0.0582 [−0.0734, −0.0426]	one seed only	No

This result does not show that interaction between the substituted amino acid and its context is absent. It shows that, at this sample size (89 clusters), what four different mechanisms add is not enough to offset the variance they bring. The question of model capacity is therefore left open and is not answered by trying one more variant (see Section 4.2.3.4).

4.2.3.4. Learning Curve and Relaxing the Degron Threshold

Two questions were put together and their criteria fixed before running. **P1:** does the AUROC still rise with the number of training clusters? **P0:** can the 85 pep-

tides removed by the degron threshold be used as extra *training* labels if the threshold is relaxed from 3.0 to 4.0? The design keeps the outer-test cells exactly equal to those of the 91 benchmark peptides, so every number is comparable with 0.8082; the added peptides enter only the outer-training sets, and those that fall in a test cluster are removed from the training of that fold (531 cells in fold 0 and 532 cells in fold 3, none in the other folds). The partition of the five folds was checked to be identical to the earlier experiment; at the 100% level, the two models reproduce the earlier scores of 0.8082 and 0.8015 exactly. Two models of different capacity were fixed in advance: ridge M3 and the gated network without ΔPAP.

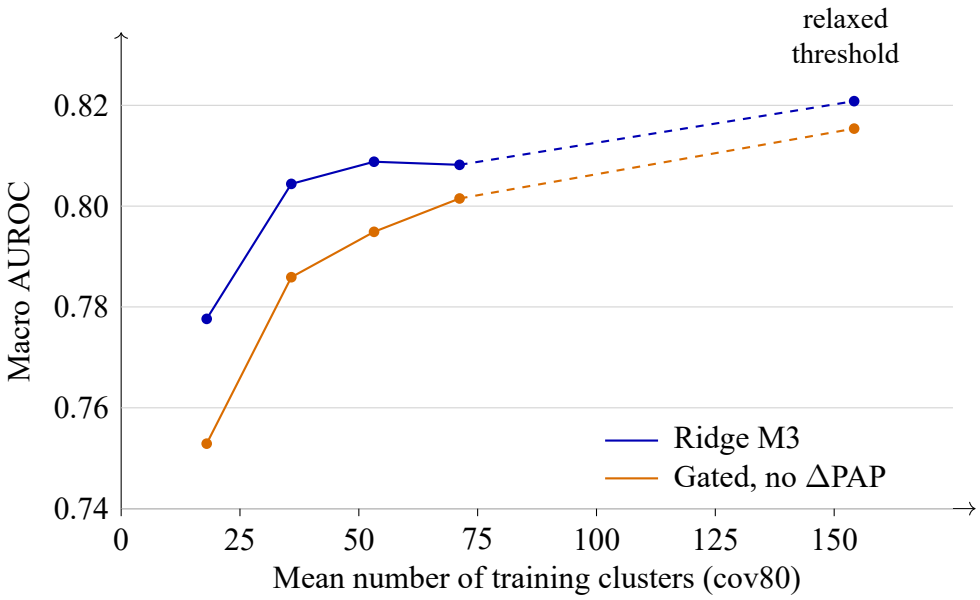


Figure 4.1. Macro AUROC against the mean number of training clusters (nested cluster subsets of 25, 50, 75 and 100% of the outer-training data; the dashed segment is the relaxed-threshold training set).

P1. Table 4.6 and Figure 4.1 show the curves. Under the fixed rule the learning curve of ridge does not support the hypothesis of label shortage (Table 4.7). From 100% to 50% of the clusters the difference is only +0.0038 ([−0.0009, +0.0087]) and the sequence of means is not monotone. This must not be read as evidence that the data limit has been reached: the design does not measure a ceiling. The gated network behaves differently: it gains +0.0156 ([+0.0060, +0.0256]), the curve is monotone, and the criterion is met. The two curves together suggest that ridge has absorbed what the original distribution of clusters can teach it, while the larger-capacity network is

Table 4.6. Learning curve (P1) and relaxed degron threshold 4.0 (P0). Ranges are over 5 folds and 3 seeds.

Training level	Train clusters	Train cells	Ridge M3	Gated, no ΔPAP
25%	18	9 068–10 314	0.7776	0.7529
50%	35–36	18 635–19 877	0.8044	0.7859
75%	52–54	28 145–29 048	0.8088	0.7949
100%	70–72	38 233–38 290	0.8082	0.8015
100% + relaxed threshold	153–155	82 191–82 780	0.8208	0.8154

still rising at about 70 clusters. The gap between ridge and the network narrows with data but is not closed (ridge is still higher by 0.0054 at the relaxed-threshold level); the result does not rehabilitate any of the rejected designs, and it must not be extrapolated to "more data will make the architecture win". One of the predictions written before the run, that P1 would probably be supported, was therefore not borne out for ridge.

P0. Adding 85 peptides (44 490 cells, 83 clusters not in the benchmark) lifts ridge from 0.8082 to 0.8208, a gain of +0.0126 ([+0.0061, +0.0190]), positive in each seed (+0.0142, +0.0142, +0.0128). The gated network gains +0.0139 ([+0.0059, +0.0218]). Both meet the pre-registered criteria. The protocol had also predicted, before the run, that P0 would probably fail, because peptides with a high PSI_{WT} are already stable and might have almost no positive cells. The measurement contradicts this prediction (Table 4.8): the share of positive cells falls only from 13.1% to 11.4%, and the 95th percentile of ΔPSI in the added peptides is 1.51, against 1.70 in the benchmark peptides. The threshold of 3.0 therefore appears more conservative than the mutation-effect task requires, even if it is the right threshold for the different question of whether a peptide is a degron.

4.2.3.5. Number of Clusters or Coverage of the Distribution

The two curves of the previous section seem to contradict each other: ridge is flat between 35 and 71 clusters, yet it rises when 83 more clusters are added. One reading is that the two steps add different things: the first thickens the existing region of PSI_{WT}, the second widens it. The previous experiment cannot separate the two, because it changes the number of clusters and the region together. A second experi-

Table 4.7. Pre-registered verdicts of P1 and P0

Model	Question	ΔAUROC [95% CI]	Pre-registered criteria	Verdict
Ridge M3	P1: 100% − 50%	+0.0038 [−0.0009, +0.0087]	CI> 0: no; monotone: no	insufficient evidence for label shortage
	P0: relaxed − 100%	+0.0126 [+0.0061, +0.0190]	CI> 0: yes; all seeds > 0: yes	accepted
Gated, no ΔPAP	P1: 100% − 50%	+0.0156 [+0.0060, +0.0256]	CI> 0: yes; monotone: yes	supported
	P0: relaxed − 100%	+0.0139 [+0.0059, +0.0218]	CI> 0: yes; all seeds > 0: yes	accepted

Table 4.8. Cells of the benchmark peptides and of the 85 peptides added by relaxing the threshold

Statistic	Benchmark set (91 peptides)	Added set (85 peptides)
Median PSI_{WT}	2.593	3.346
Fraction of positive cells ($\Delta PSI \geq 1$)	0.1310	0.1140
Median ΔPSI	0.125	0.083
95th percentile of ΔPSI	1.697	1.507

ment, with its criteria fixed before running, keeps the whole original training set and adds a nested fraction of the new clusters (Table 4.9). The main criterion compares the gain at +50% of the new clusters (about 42 clusters) with the marginal gain of 36 old-region clusters measured before, 0.0038.

Table 4.9. Gain over the benchmark training set when a fraction of the new clusters is added

New clusters added	Mean added	Ridge	Gain	Gated	Gain
none (benchmark)	0	0.8082	—	0.8015	—
+25%	21	0.8143	+0.0061	0.8090	+0.0075
+50%	42	0.8196	+0.0114	0.8118	+0.0103
+75%	64	0.8204	+0.0122	0.8168	+0.0153
+100%	85	0.8208	+0.0126	0.8154	+0.0139

For ridge the gain at +50% is +0.0114 ([+0.0063, +0.0166]), whose interval lies entirely above 0.0038: the main criterion supports coverage of the distribution. For the network the interval contains 0.0038, so the new clusters are useful but not shown to be better than old ones (Table 4.10). The secondary criterion, the shape of the curve, gives the opposite answer: the gain at +25% is 48.4% of the total for ridge and 54.4% for the network, below the threshold of 60% fixed for "early saturation", so the fixed rule says "proportional to the number of clusters". Because the two fixed criteria disagree, the question is *not decided*. As a descriptive remark only, the successive increments of ridge are +0.0061, +0.0053, +0.0007, +0.0005, so roughly 90% of the total gain is already reached at +50%, and the threshold of 60% at +25% was a poor operational choice. This remark does not overturn the verdict.

What can be stated is limited: clusters from the region $3.0 < \mathrm{PSI}_{\mathrm{WT}} \leq 4.0$ have a marginal value at least equal to that of old-region clusters, and for ridge the evidence suggests it is higher. It cannot be said that the gain is *only* due to wider coverage, because the 83 new clusters differ from the old ones both in region and in protein.

4.2.3.6. Sensitivity to the Threshold

The four thresholds 3.5, 4.0, 4.5 and 5.0 were run in one pass and are all reported (Table 4.11, Figure 4.2). The main configuration of the thesis stays at 4.0, a value fixed before this grid was seen and chosen then only because it roughly doubles

Table 4.10. Pre-registered verdicts on the new clusters. The reference marginal gain of old-region clusters is 0.0038.

Model	B(50%) − B(0) [95% CI]	Share of total gain at +25%	Primary criterion	Shape criterion
Ridge M3	+0.0114 [+0.0063, +0.0166]	48.4%	supports wider coverage of the distribution	proportional to cluster count
Gated, no ΔPAP	+0.0103 [+0.0033, +0.0172]	54.4%	valuable, but not shown to exceed old-region clusters	proportional to cluster count

the number of peptides; it is not re-chosen from the grid. Every threshold is better than the benchmark training set (all intervals above zero). The ridge gain is highest at 4.0 (+0.0126) and lower at 4.5 (+0.0101) and 5.0 (+0.0093); under the fixed rule (a drop of more than 0.002) this is labelled "decline after 4.0". That label describes the shape of the point estimates: the intervals overlap widely, and no interval was computed for the difference between 4.0 and 4.5. The gated network is nearly flat from 3.5 to 5.0, so the decline is a property of ridge. It also depends on how a positive cell is defined: with the positive cut at $\Delta\text{PSI} \geq 1.5$ the maximum for ridge moves to 5.0 (+0.0098) and there is no decline. The share of positive cells in the added peptides falls steadily with the threshold (Table 4.11), which is consistent with peptides that are not degrons starting to dilute the training set (an interpretation, not tested separately).

Table 4.11. Added data and gains for each degron threshold (cov80, positive cut $\Delta\text{PSI} \geq 1$)

Thr.	Peptides	Cells	New clusters	Pos. rate	Ridge AUROC	Ridge Δ [95% CI]	Gated Δ
3.5	+50	26 234	48	0.1197	0.8185	+0.0103 [+0.0043, +0.0164]	+0.0115
4.0	+85	44 490	83	0.1140	0.8208	+0.0126 [+0.0061, +0.0190]	+0.0139
4.5	+106	55 362	102	0.1028	0.8183	+0.0101 [+0.0028, +0.0172]	+0.0125
5.0	+121	63 009	117	0.0918	0.8175	+0.0093 [+0.0016, +0.0169]	+0.0133

The gain at 4.0 survives three definitions of a positive cell and a different assignment of folds (Table 4.12); all four intervals are above zero. The last row must

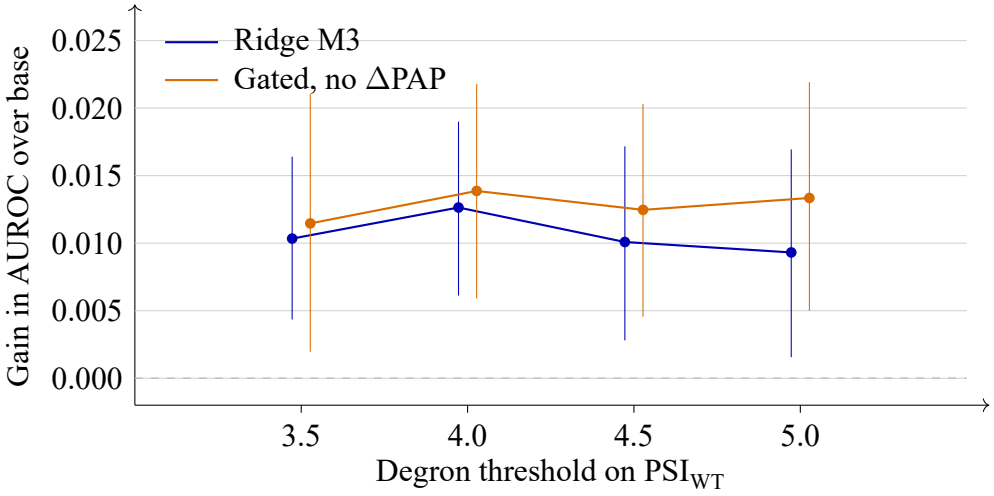


Figure 4.2. Gain in macro AUROC over the benchmark training set for each degron threshold, with 95% cluster-bootstrap intervals (horizontal offsets only separate the two models).

be read with care. The clustering at 50% coverage was meant to test sensitivity to the coarseness of clustering, but on this peptide set cov50 and cov80 produce the same number of clusters (89) and, as found after the run, the same grouping of peptides; only the representative labels differ, which changes how clusters are assigned to the five folds (the benchmark score becomes 0.8116 instead of 0.8082). That row therefore tests a re-assignment of folds, not coarser clustering; clustering granularity is tested with genuinely different partitions in Section 4.2.3.7.

Table 4.12. Robustness of the gain at threshold 4.0 (ridge M3)

Condition (threshold 4.0)	Ridge Δ vs. base [95% CI]	CI > 0
cov80, ΔPSI ≥ 0.5	+0.0110 [+0.0051, +0.0169]	yes
cov80, ΔPSI ≥ 1.0 (main)	+0.0126 [+0.0061, +0.0190]	yes
cov80, ΔPSI ≥ 1.5	+0.0094 [+0.0037, +0.0149]	yes
cov50 fold assignment, ΔPSI ≥ 1.0	+0.0096 [+0.0032, +0.0159]	yes

4.2.3.7. Sensitivity to the Clustering Granularity

Because cov50 does not change the partition on this peptide set, a grid of seven partitions of the 176 peptides (the 91 benchmark peptides and the 85 added ones) was fixed before running: grouping by UniProt accession (no clustering), the reference MMseqs2 setting (30% identity, 80% coverage) and five looser MMseqs2 settings.

The P0 comparison (ridge M3, relaxed threshold 4.0 against the benchmark training set, same cluster bootstrap) was repeated with the folds rebuilt from each partition. The reference level reproduces the earlier scores (0.8082 and 0.8208) and the five folds of the earlier experiment. The criterion fixed in advance: the gain is "robust to granularity" if every partition that differs from the reference and is still informative (fewer than half of the added cells are lost because they share a group with a test peptide) has a bootstrap interval of the gain above zero and a positive gain in all three seeds.

Table 4.13. Gain of the relaxed threshold 4.0 (ridge M3) under different protein partitions. "Groups" are the groups of the 91 benchmark peptides and of all 176 peptides; "Shared" is the number of added peptides that fall in a group of a benchmark peptide; "Excl." is the mean percentage of the added cells that are removed from training because their group contains a test peptide.

Partition (MMseqs2)	Groups base/all	Shared	Excl. %	AUROC base → relaxed	Gain [95% CI]	Seeds > 0
Accession (no clustering)	91/176	0	0.0	0.8036 → 0.8202	+0.0166 [+0.0099, +0.0240]	yes
id 0.3, cov 0.8 (reference)	89/172	2	0.5	0.8082 → 0.8208	+0.0126 [+0.0061, +0.0190]	yes
id 0.3, cov 0.5 grouping as reference; folds assigned differently	89/172	2	0.5	0.8116 → 0.8212	+0.0096 [+0.0032, +0.0159]	yes
id 0.2, cov 0.5	87/163	6	1.4	0.8051 → 0.8185	+0.0134 [+0.0074, +0.0199]	yes
id 0.1, cov 0.5 identical to the row above	87/163	6	1.4	0.8051 → 0.8185	+0.0134 [+0.0074, +0.0199]	yes
id 0.1, cov 0.3	71/133	22	5.2	0.8096 → 0.8209	+0.0113 [+0.0037, +0.0184]	yes
id 0.0, cov 0.3 identical to the row above	71/133	22	5.2	0.8096 → 0.8209	+0.0113 [+0.0037, +0.0184]	yes

The seven levels give only 4 distinct partitions (Table 4.13): lowering the identity from 0.2 to 0.1 at coverage 0.5, and from 0.1 to 0.0 at coverage 0.3, changes no group, and 30% identity with coverage 0.5 reproduces the reference grouping (only

the group labels, and hence the fold assignment, differ; the gain there is +0.0096 [+0.0032, +0.0159], the same value as the cov50 row of Table 4.12). Three partitions differ from the reference and are informative: grouping by accession (+0.0166, [+0.0099, +0.0240]), 20% identity with coverage 0.5 (+0.0134, [+0.0074, +0.0199]) and 10% identity with coverage 0.3 (+0.0113, [+0.0037, +0.0184]). All intervals are above zero and the gain is positive in all three seeds, so under the fixed rule the gain is *robust to granularity*. The point estimates are not monotone in the coarseness (the largest is for grouping by accession), the intervals overlap, and no test of their differences was planned; the absolute benchmark score also moves with the partition (0.8082 for the reference and 0.8116 for the same grouping with other folds), so absolute values must not be compared across rows.

The test is weaker than its name suggests. The coarsest partition still has 71 groups for the 91 benchmark peptides; in it, 22 added peptides share a group with a benchmark peptide and on average 5.2% of the added cells are removed from training, and one of the five test folds consists of a single group. Whether the gain would survive a still coarser split, for example by protein family, is not known.

4.2.3.8. Pre-Training on Tile Labels

The experiment of Voutsinos et al. [4] provides a degradation score for 212 616 tiles of 30 residues, without substitutions. The question was whether initialising the gated network (without ΔPAP) from a model pre-trained to predict these scores lets it learn ΔPSI better under the same label budget. The design was fixed in advance. Pre-training treats each tile as 30 wild-type rows (the substituted-residue input is zero), predicts the tile as the mean over positions and regresses the sign-reversed, standardised degron score. Two branches then share the same initial weights, the learning-rate grid (2×10^{-3} or 5×10^{-4}, chosen on the inner validation set) and the inner validation split, and differ only in whether the network starts from the pre-trained state (*pre*) or from random initialisation (*rand*). To prevent leakage, in each outer fold every gene that has a tile similar to a protein of an outer-test cluster (MMseqs2 at 30% identity with an alignment of at least 50 residues, a shared 12-mer, or the same gene name) is removed entirely from the pre-training data *before* any learning; this removed 318, 34, 176, 99, 44 genes in the five folds. The gate on the reproduction of earlier scores was passed (ridge 0.8082 and 0.8208; the gated network within 6.0e-05 of the earlier

values). Two comparisons were fixed, both with the rule "bootstrap interval above zero and a positive difference in every seed": (A) *pre* against *rand*, and (B) *pre* against ridge M3.

Table 4.14. Macro AUROC with and without pre-training on tile labels

Training configuration	Ridge M3	Network, random init.	Network, pre-trained
Relaxed threshold 4.0 (main)	0.8208	0.8175	0.8098
Threshold 3.0 (secondary)	0.8082	0.8009	0.7726

Table 4.15. Pre-registered comparisons for pre-training. The main configuration is the relaxed threshold 4.0; the threshold 3.0 is reported with the same criteria and was not used to choose the configuration.

Thr.	Comparison	Difference [95% CI]	Per-seed difference	Met
4.0 (main)	A: pre-trained − random init.	−0.0077 [−0.0163, +0.0006]	−0.0180, −0.0259, −0.0121	no
	B: pre-trained − ridge M3	−0.0110 [−0.0197, −0.0024]	−0.0234, −0.0330, −0.0263	no
3.0 (secondary)	A: pre-trained − random init.	−0.0283 [−0.0426, −0.0149]	−0.0295, −0.0345, −0.0427	no
	B: pre-trained − ridge M3	−0.0356 [−0.0509, −0.0215]	−0.0458, −0.0550, −0.0637	no

The result is negative (Tables 4.14 and 4.15). In the main configuration the pre-trained network reaches 0.8098, against 0.8175 for random initialisation: difference A is −0.0077 [−0.0163, +0.0006], an interval that contains zero, so pre-training cannot be distinguished from random initialisation; difference B is −0.0110 [−0.0197, −0.0024], so the pre-trained network is below ridge. At threshold 3.0 both differences are clearly negative (A: −0.0283 [−0.0426, −0.0149]; B: −0.0356 [−0.0509, −0.0215]). The difference is negative in all three seeds in every comparison. By the mapping fixed in advance, the conclusion is "insufficient evidence for a benefit of pre-training". The pre-training task itself was learned (Pearson correlation 0.864 between prediction and score on held-out genes, range 0.857–0.868 over the fifteen runs), so the failure is in the transfer to ΔPSI and not in learning the tile scores. This result rejects only this design, in which the pre-training sees wild-type sequences only and the whole network is then fine-tuned; designs that pre-train with substitutions, freeze

part of the network or train both tasks jointly were not tried, and nothing is inferred about them.

4.2.3.9. Calibration of the Scores and Ranking Utility

The score of ridge M3 is a predicted ΔPSI, not a probability. To report the probability that a substitution is positive ($\Delta\text{PSI} \geq 1$) the model trained at the relaxed threshold 4.0 was kept fixed and evaluated on the same 47 847 outer-test cells (91 peptides, 89 clusters). Calibrators were fitted only on out-of-fold scores of the outer-training pool: the pool was split again into five inner folds by cluster, the ridge model was refitted on four of them, and the scores of the fifth were kept; the outer-test cells never entered a calibrator. Three conversions were fixed in advance: the *raw* score clipped to $[0, 1]$, Platt scaling (a logistic regression on the score) and isotonic regression, with a constant forecast of the base rate of the training pool as a reference for skill. The criteria were also fixed in advance: a conversion is *calibrated* if the bootstrap interval of $\text{Brier}(\text{raw}) - \text{Brier}(c)$ is above zero and the difference is positive in every seed; it has *skill over the base rate* if the interval of $\text{Brier}(\text{constant}) - \text{Brier}(c)$ is above zero; Platt is used in the prototype unless isotonic regression is better with an interval above zero; and the ranking is *enriching* if the interval of the enrichment at 10% is above 1. The raw scores of the three seeds reproduce the earlier AUROC values (gate passed).

Table 4.16. Probability quality on the outer-test cells (observed fraction of positive cells: 0.131). ECE: expected calibration error over ten equal-mass bins; $\hat{p}$: predicted probability.

Score to probability	Brier	95% CI	Skill vs. base rate	ECE	Mean $\hat{p}$
Raw score, clipped to $[0, 1]$	0.1272	[0.1186, 0.1360]	−0.116	0.1437	0.2617
Platt (logistic) scaling	0.0925	[0.0809, 0.1046]	+0.188	0.0163	0.1146
Isotonic regression	0.0925	[0.0810, 0.1045]	+0.188	0.0164	0.1145
Constant (base rate of the training pool)	0.1139	[0.1011, 0.1276]	—	0.0154	0.1219

Platt scaling meets the criterion (Tables 4.16 and 4.17): the Brier score falls from 0.1272 to 0.0925, a difference of +0.0346 [+0.0266, +0.0425], positive in each

Table 4.17. Pre-registered comparisons of the Brier score

Comparison (positive favours the second term)	Difference [95% CI]	Per-seed difference
Brier(raw) − Brier(Platt)	+0.0346 [+0.0266, +0.0425]	+0.0346, +0.0347, +0.0347
Brier(raw) − Brier(isotonic)	+0.0346 [+0.0267, +0.0424]	+0.0345, +0.0347, +0.0347
Brier(constant) − Brier(Platt)	+0.0214 [+0.0175, +0.0253]	+0.0212, +0.0215, +0.0215
Brier(Platt) − Brier(isotonic)	+0.0000 [−0.0002, +0.0002]	—

seed (+0.0346, +0.0347, +0.0347); isotonic regression is practically identical (Platt − isotonic: +0.00002, [−0.0002, +0.0002]), so Platt scaling, with two parameters, is used. The skill over the base rate is +0.188, with the interval of +0.0214 [+0.0175, +0.0253] above zero. The first criterion is a low bar: the raw score read as a probability is *worse* than a constant forecast (skill −0.116; mean value 0.262 against 0.131 observed; recalibration slope 0.35), because it is a regression output. The more informative statement is the skill of about +0.188. The ECE alone does not separate the methods here, since the constant forecast has an ECE (Table 4.16) close to that of Platt scaling.

Local calibration is not perfect. The Platt probabilities are lower than the observed frequency in every bin: the mean is 0.115 against 0.131, and in the highest bin the mean prediction is 0.434 against an observed 0.495 (interval 0.449–0.538). A likely cause is a shift of domain introduced by the relaxed threshold: the calibrator is fitted on a pool that includes the added peptides, in which the fraction of positive cells is lower than in the benchmark peptides (Table 4.8). A sensitivity analysis that was not used for the verdict, fitting the calibrator only on benchmark-domain cells, gives a Brier score of 0.0921, an ECE of 0.0044 and a mean probability of 0.131. For ordering substitutions the shift is irrelevant (the transform is monotone); for absolute probabilities it must be kept in mind.

For prioritising substitutions (Table 4.18), choosing the top 10% of each peptide (about 54 substitutions) gives a precision of 0.47 and a recall of 0.41, an enrichment of 4.02 ([3.70, 4.35]); the criterion is met. The only reference is a random ranking: it is not known how much better the model is than a simple rule, for example

Table 4.18. Ranking utility: the substitutions of each peptide (532 per 28-residue peptide) are ordered by the raw score. A random ranking has enrichment 1.

Top fraction	Cells	Precision	Recall	Enrichment	95% CI
5%	27	0.551	0.241	4.75	[4.23, 5.28]
10%	54	0.466	0.407	4.02	[3.70, 4.35]
20%	107	0.371	0.611	3.04	[2.86, 3.21]

one based on the hydrophobicity of the substituted residue. Moreover, the final model is fitted on all of the pool whereas the calibrator is learned from scores of models fitted on four fifths of it (a known limitation of cross-fitting), and all of this is still a development benchmark. The word "calibrated" here means that the Brier score is better than that of the raw score with an interval above zero; it does not mean perfect calibration.

4.2.3.10. External GPS Labels: Koren et al. Table S4

The saturation-mutagenesis tables of Koren et al. [2] use the same endpoint (PSI) and were audited as a possible source of additional training labels. The supplementary tables were downloaded manually, because the automatic download is blocked by the host. The article is not in the open-access subset of the repository, so the tables are used internally only and are not redistributed here. The gate fixed in advance required all of four conditions (Table 4.19).

Table 4.19. Gate fixed in advance for merging the labels of Koren et al. (Table S4) into training

Criterion	Pre-registered threshold	Observed	Met
G1: new clusters at cov80	≥ 20	5 (6 of 13 sequences mapped)	No
R1: PSI recomputed from bin counts	$\lvert\Delta\rvert \leq$ 1e-06 in $\geq 99\%$ of rows	1827 of 1827 rows (max. 5.0e-12)	Yes
R2: internal wild-type reference	≥ 8 synonymous cells in ≥ 7 substrates	9 of 9 substrates	Yes
R3: direction of ΔPSI (last Gly of Clone#8)	median $\geq +0.5$, $\geq 90\%$ positive	median +1.04, 100% positive ($n = 19$)	Yes
Gate (G1 and R1 and R2 and R3)			**No**

The data themselves are sound: PSI is reproduced from the bin counts in all

rows, an internal wild-type reference can be built for all substrates, and the direction of the effect of the last glycine agrees with the biology. The gate nevertheless fails on the number of new clusters. Table S4 holds 1 739 substitution cells for 9 substrates; the peptides are 23-residue C-terminal sequences with a stop codon in which only the last ten positions are mutated, a library different from the 28-residue internal peptides of the main labels; PSI is computed from 3 bins and not 6, so ΔPSI values from the two sources cannot simply be added; only 3.0% of the cells have $\Delta\text{PSI} \geq 1$ (against 13.1% in the benchmark), and the effects sit in the last one to three positions. Only 6 of the 13 sequences could be mapped to Swiss-Prot, and one of them overlaps with the training peptides, which leaves 5 new clusters, against the 20 required. As a diagnostic, not part of the gate, the agreement between Tables S3 and S4 on the 12 cells they share is low (Spearman 0.10), so S3 is not used as a label source.

By the consequence fixed in advance, these labels are *not* merged into training, and the planned merging of external labels (P4) and the repeated learning curve on the enlarged set (P5) were therefore not run. The table remains a possible external test set for a C-terminal degron class; this was not done in the thesis and no number is reported for it.

4.2.3.11. Scope Comparison with MetaDegron

MetaDegron [8] was audited as a possible competitor and label source. Its labels are binary motif-level annotations of curated degrons, not ΔPSI; the audit therefore compares scope and data, not predictive scores on the GPS matrix, because MetaDegron-D could not be run on GPS peptides (Table 4.20). Among the main findings, the curated positive set has only 227 human rows, and almost no region of it coincides with the GPS peptides used here (2 scanning peptides, 1 saturation peptide and 0 from the CRISPR screens). Only 1 E3 motif group keeps at least five samples after the strictest filter, so “21 E3 ligases” is a list of motif-compatible assignments, not 21 independently labelled classes. The reported MetaDegron-X AUC of 0.9670 was recomputed from the stored trees, but only on the authors’ prepared tables, and 8 of the 20 positive test coordinates are in the property table of the fitting data; it is not an independent result and not a GPS result. A simple composition logistic baseline on the same positives, split by protein cluster at cov80, gives AUROC between 0.728 and 0.731 (core composition) and between 0.737 and 0.747 (29-residue context), on a

reconstructed population and not on the authors' own training set. The audit therefore suggests positioning MetaDegron as a complementary tool for motif and E3 information rather than a head-to-head competitor on residue-level effect prediction; binary motif labels are not converted into ΔPSI.

Table 4.20. Audit of the MetaDegron data and overlap with the data of this thesis

Audit item	Value
Curated positive rows / distinct accessions	302 / 240
Rows with species suffix HUMAN	227
Rows in the property table used by MetaDegron-X	212
E3 motif groups with ≥ 5 samples: before filtering / after the strictest filter (human only, no regional or cluster overlap)	13 / 1
Same-region overlap with GPS scanning peptides (2 624 peptides)	2
Same-region overlap with saturation peptides (248 mapped)	1
Same-region overlap with CRISPR-screen peptides (31 peptides)	0
MetaDegron-X AUC recomputed from the stored trees on the authors' prepared tables (not a GPS result)	0.9670
Positive test coordinates also present in the property table of the fitting data	8 / 20

4.2.3.12. Experiments Not Done

The following parts of the plan have no results in this version of the thesis; no number is given for them.

- **Merging external GPS labels and repeating the learning curve on the enlarged set.** Not run: the only audited source (Table 4.19) did not pass its gate. Other candidate sources (for example Timms et al., Shimshon et al., Bekturova et al. and a recent preprint on disease mutations that disrupt degrons) have not been audited, and their terms of use have not been read.

- **Independent validation.** All numbers are out-of-fold on a development benchmark. Testing on labels that did not take part in any design decision, including the C-terminal data of Table 4.19, has not been done. **[PENDING]**

- **Other pre-training designs** (with substitutions, partial freezing, joint training): not tried (Section 4.2.3.8).

- **Comparison of the ranking with simple rules.** Only a random ranking was used as reference (Section 4.2.3.9).
- **Separating the region of PSI_{WT} from the protein.** It needs pairs of peptides of the same protein on both sides of the threshold, which the present data may not contain (Section 4.2.3.5).
- **Expanded local software acceptance and tests with users.** Baseline web/API code exists, but project/history, batch/protein and import/comparison extensions and their acceptance remain pending (Section 4.3); historical checks do not verify this expanded scope.

4.2.4. Discussion

What the measurements support. (i) A ridge regression on 180 features is a strong baseline for the substitution effect: it is above the frozen ΔPAP baseline with an interval that excludes zero, and none of the five larger designs tried (the sigmoid-gated networks, FiLM, the bilinear form, the cross-product ridge and the retrained transformer architecture) beat it under the fixed criteria. (ii) Training labels from peptides just above the degron threshold improve the out-of-fold AUROC on the original test cells, and this gain is not caused by an obvious collapse of the label. (iii) The gain is not simply proportional to the number of clusters, but the data do not yet separate wider coverage from more clusters. (iv) The gain from the relaxed threshold does not depend on how proteins are grouped: it is above zero under every informative partition tried, including grouping by accession. (v) The score of ridge M3 can be turned into a probability with a Platt transform that is better than the raw score and than the base rate, and the top 10% of the substitutions of a peptide contains about four times more positive cells than a random choice.

What they do not support. The thesis does not claim a new architecture: no architectural component earned a place under the fixed criteria, and the gate even changes sign between seeds. It does not claim that the published Degformer is worse than the models here, nor better: its checkpoint is not a held-out result, and the retrained architecture was run for one seed. It does not claim that more labels make a larger model win; the gated network is still rising, but it is below ridge at every level that

was run. It does not claim that pre-training on tile labels helps: the one design tried was not better than random initialisation and was below ridge. It does not claim that the external labels of Koren et al. can be merged: they did not pass the gate. It makes no claim of external validation of the probabilities or of the ranking.

Threats to validity.

- The benchmark is a development benchmark on a data set that has been examined several times; the out-of-fold numbers are not an external validation, and the best figure (0.8208) must not be quoted as one.
- The three seeds share one outer partition, and the confidence intervals are conditional on the stored predictions. No multiple-comparison correction is applied.
- The threshold of 4.0 was fixed before the sensitivity grid, but only after the benchmark and a first look at the number of peptides it adds; the near coincidence with the maximum of the grid is not evidence that the way it was chosen was sound.
- All labels come from one assay and one library. Whether the gain transfers to other degron classes or to other assays is untested.
- The added peptides pass through the same label pipeline and so inherit its assumptions (the PSI_{WT} reference from synonymous cells, the average of two halves).
- The coarsest partition tested still has 71 groups for 91 peptides, so robustness to a split by protein family is untested; the absolute scores change with the partition.
- The Platt calibrator is fitted on a pool that includes the added peptides, and its probabilities are about 13% too low for the benchmark peptides; the criterion “better than the raw score” is a low bar, because the raw score is worse than the base rate.
- The pre-training result concerns one design and one encoder; the ranking is compared only with a random ranking.

4.2.5. Reproducibility of the Reported Results

Tables, figures and numerical summaries are generated from the recorded experimental outputs using consistent filtering, aggregation and rounding rules. The experimental setup specifies the data, protein-cluster partitions, seeds and evaluation metrics; each result is interpreted with the controls and limitations of its experiment. The underlying records retain the configurations and input identities needed for computational reproduction. These internal records support traceability, while the thesis presents the scientific methods, results and their interpretation.

4.3. Software Testing Plan

4.3.1. Functions Under Test

The target covers UC-01 to UC-10 of Chapter 1. Functions include scoring, ranking and export; protein selection and ClinVar annotations; model and reference evidence; batch and protein jobs; experimental import and comparison; and persistent projects and history. The recorded CLI verification below covers numerical consistency for the original scoring and export path only. Baseline web/API code and historical test records exist, but these tests have not been rerun here and do not verify the new workflows. The expanded local acceptance scenarios are specified later in this chapter.

4.3.2. Approach

The prototype is a thin layer over a research model, so the main risk is that it computes something different from what was evaluated in Section 4.2. The approach is therefore *back-to-back testing*: the prototype path is compared with the training and evaluation path of the research runner on the 92 337 cells of the 176 peptides. The 180 features are rebuilt in the prototype by an independent implementation (16 620 660 values are compared), so the comparison is not a code path checked against itself. Baseline input-validation tests exist in the application source. Their historical results are not new execution evidence in this chapter. New domain, job, import, annotation and history cases below remain pending implementation and execution.

4.3.3. Criteria for Test Success or Failure

A check passes when the largest absolute difference between the two paths is below 10^{-9}. The check of the exported file is asserted at 5×10^{-9} in the code, because values are written with ten significant digits; the measured differences are below 10^{-9} for it as well. A test fails when any difference exceeds its limit. For the new workflows, a functional case also requires the specified item states, aligned counts, warnings and durable/deleted records. A skipped or pending case is not a pass; compatible-metric fixtures need independently calculated answers.

4.3.4. Suspension Criteria and Restart Requirements

The verification script stops at the first failed assertion and runs no later check. The command-line tool refuses to run if the model file is missing. Restart: rebuild the frozen model artefact and run the verification procedure again from the beginning.

4.3.5. Test Deliverables

The verification procedure, its machine-readable execution report, the calibration results and Table 4.22 of this chapter form the recorded numerical-consistency evidence. The pending local acceptance work additionally requires dated functional, recovery and resource-use reports.

4.4. Test Management

4.4.1. Test Activities

Executed: building the model and the caches, running the verification script (11 checks). Not executed for the expanded first-release scope: local integration, job/import/history tests, resource measurement and user evaluation. Historical baseline web/API checks are a separate record.

4.4.2. Test Environment

Linux under WSL2 on one development computer, CPU only; Python 3.12.3 and NumPy 2.5.3. The verification needs about 1.2 GB of memory. The model file and the caches are generated by the scripts and are not stored in the repository.

4.4.3. Resources and Allocation

One tester, the author of the thesis, who is also the developer; there is no independent tester.

4.4.4. Risks

- Back-to-back tests show agreement with the research code, not that the science is right: an error shared by both paths (for example in the definition of the label) would not be found.

- The probabilities are about 13% too low for peptides of the original benchmark domain (Section 4.2.3.9); ranking is not affected.

- The historical CLI has only a printed domain warning. The baseline web gate still needs R3 integration; whole-protein processing must not hide unknown, borderline or out-of-domain tiles.

- User imports can mix endpoints or mis-map residues; validation and separate assay comparisons are required. Persistent jobs/history add crash recovery and file-cleanup risks.

- The model was evaluated only on a development benchmark; no result in this chapter supports a claim of external validity.

4.5. Test Scenarios and Test Cases

4.5.1. Test Scenarios

S1: score a tile given as a sequence. S2: score a tile of a protein given by its UniProt accession. S3: export the scores as CSV and read them back. S4: reject invalid input; the retained CLI verification report does not cover this scenario.

4.5.2. Expanded Local Acceptance Scenarios

Table 4.21 defines the additional scenarios. All are pending; test IDs map to the system SRS and test plan. No public-host deployment is required.

Table 4.21. Planned first-release local acceptance scenarios; no execution result claimed

Case group	Expected evidence
DOM-01, ANN-01	Correct gate estimator/borderline handling; ClinVar and reference mapping, source states and attribution.
BATCH-01 to 06	Bounds, single-tile equivalence, coverage/overlapping contexts, partial failures, cancellation, restart without duplicates and complete exports.
IMP-01 to 04	CSV/TSV validation/commit, WT/replicate alignment, missing/conflicting data and hand-checked assay-compatible metrics.
HIST-01 to 03	Explicit local persistence, immutable reopen/rerun after source changes, confirmed dependency/file deletion.
NF-07, DEP-01/02	Recorded local resource use, clean local installation and restart of the same instance.

4.5.3. Test Cases

Table 4.22 lists the executed cases, which cover S1 to S3 through the comparison of the scores computed by the prototype with those of the research path. The cases of S4 (length different from 28, a character outside the 20 amino acids, an unknown accession, a tile beyond the end of the protein, a tile with a non-standard residue, a missing model file) are specified by the CLI checks; application source contains baseline tests, but no new execution report is added for them here. Expanded first-release cases are specified in Table 4.21.

4.6. Test Evaluation

All 11 executed cases passed; the largest absolute difference is 4.4e-10 (Table 4.22). The prototype therefore reproduces the research model on 92 337 cells, including the scores of the outer-test folds of all fifteen (seed, fold) pairs. This establishes consistency only. It does not show that the predictions are correct (Section 4.2 addresses that, on a development benchmark), and it says nothing about input validation, the interface, the response time (the target of Chapter 1 has not been measured) or usability **[PENDING]**.

The first-release software acceptance remains pending. Existing CLI measurements cannot verify project persistence, experimental-data comparison, ClinVar mapping states, whole-protein coverage or job recovery. Local acceptance must report these cases and the actual computer/resource conditions; public deployment, Internet

Table 4.22. Executed test cases: agreement of the prototype with the research runner. The tolerance is 10^{-9}.

Test case (back-to-back consistency check)	Max. abs. difference	Passed
TC-01: Labels: label pipeline vs. cached labels	0.0e+00	Yes
TC-02: Features: independent featuriser in the prototype vs. cached matrix	1.1e-16	Yes
TC-03: Raw score: prototype vs. trained ridge model	4.4e-16	Yes
TC-04: Platt probability: prototype vs. scikit-learn	3.3e-16	Yes
TC-05: Isotonic probability: prototype vs. scikit-learn	1.7e-13	Yes
TC-06: UniProt accession $\rightarrow$ tile $\rightarrow$ score: vs. training path	0.0e+00	Yes
TC-07: UniProt accession $\rightarrow$ tile $\rightarrow$ probability: vs. training path	0.0e+00	Yes
TC-08: Exported CSV, raw score: vs. in-memory prediction	4.4e-10	Yes
TC-09: Exported CSV, probability: vs. in-memory prediction	4.8e-11	Yes
TC-10: Features of the 91 benchmark peptides: prototype vs. cache	1.1e-16	Yes
TC-11: Outer-test scores of all 15 (seed, fold) pairs: prototype path vs. runner	4.4e-16	Yes

availability and hosted performance are outside the required scope.

PART 3. CONCLUSION

1. Achieved Results

1.1. Knowledge and Skills

[PENDING: to be written from the results actually achieved.]

1.2. Program and Practical Application Potential

The accepted software target is one local React/FastAPI application with peptide/protein jobs, ClinVar/reference annotations, experimental comparison and persistent projects/history. Baseline single-tile code exists; the expanded workflows still require implementation and local acceptance. This scope statement is not a claim that the first release or the research gates are complete.

2. Limitations and Challenges

[PENDING]

3. Future Work

[PENDING]

REFERENCES

[1] A. Varshavsky, "N-degron and C-degron pathways of protein degradation," *Proceedings of the National Academy of Sciences*, vol. 116, no. 2, pp. 358–366, 2019, TO VERIFY.

[2] I. Koren *et al.*, "The eukaryotic proteome is shaped by E3 ubiquitin ligases targeting C-terminal degrons," *Cell*, vol. 173, no. 7, pp. 1622–1635, 2018, TO VERIFY.

[3] Z. Zhang *et al.*, "Elucidation of E3 ubiquitin ligase specificity through proteome-wide internal degron mapping," *Molecular Cell*, vol. 83, 3377–3392.e6, 2023. DOI: 10.1016/j.molcel.2023.08.022.

[4] V. Voutsinos *et al.*, "A near-complete map of human cytosolic degrons and their relevance for disease," *Science Advances*, 2026. DOI: 10.1126/sciadv.adz3483.

[5] H.-C. S. Yen, Q. Xu, D. M. Chou, Z. Zhao, and S. J. Elledge, "Global protein stability profiling in mammalian cells," *Science*, vol. 322, no. 5903, pp. 918–923, 2008, TO VERIFY.

[6] F. B. Larsen *et al.*, "Comprehensive degron mapping in human transcription factors," *bioRxiv*, 2025, Preprint. DOI: 10.1101/2025.05.16.654404.

[7] Alpine2048, *Degformer: Transformer model for internal and C-terminal degron prediction*, GitHub repository, commit 49ee52459967, 2026. [Online]. Available: https://github.com/Alpine2048/Degformer.

[8] M. Zheng *et al.*, "MetaDegron: multimodal feature-integrated protein language model for predicting E3 ligase targeted degrons," *Briefings in Bioinformatics*, vol. 25, no. 6, bbae519, 2024. DOI: 10.1093/bib/bbae519.

[9] DEGRONOPEDIA, *DEGRONOPEDIA documentation*, Web resource, TO VERIFY: original publication. [Online]. Available: https://degronopedia.com/degronopedia/documentation.

[10] DegronMD, *DegronMD: a resource of degrons and variants*, Molecular Biology and Evolution, TO VERIFY: authors, title, year. DOI: 10.1093/molbev/msad253.

[11] M. Steinegger and J. Söding, “MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets,” *Nature Biotechnology*, vol. 35, no. 11, pp. 1026–1028, 2017, TO VERIFY.

[12] A. E. Hoerl and R. W. Kennard, “Ridge regression: biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970, TO VERIFY.

[13] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30*, TO VERIFY, 2017.

[14] B. Efron, “Bootstrap methods: another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979, TO VERIFY.

[15] Z. Zhang *et al.*, *Data accompanying the proteome-wide internal degron mapping study*, Mendeley Data, version 2, 2023. DOI: 10.17632/32h94v2yv2.2.

[16] NCBI, *GEO Series GSE240610*, Gene Expression Omnibus. [Online]. Available: https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE240610.

APPENDIX A: INSTALLATION AND USER GUIDE

A.1. Installation Requirements

A.1.1. Hardware Requirements

CPU inference runs on one local computer. Record actual OS, CPU, RAM and free disk in the local test report; batch/import/project storage requirements remain to be measured. A GPU is not required by the runtime application. Offline research training has separate requirements.

A.1.2. Software Requirements

The baseline uses a Python virtual environment and pinned backend dependencies, Node/npm with the frontend lockfile, and locally prepared reference data/model weights. The target adds local PostgreSQL and a job worker. Exact versions are recorded by the implementation's requirements/lockfiles; PostgreSQL/job integration is still pending. The research runner additionally uses its own packages and MMseqs2; these are not all required for serving predictions.

A.2. Local System Execution

There is one local web application. Public hosting, VPS, domain and TLS deployment are outside the required workflow. All application listeners must be bound to loopback.

A.2.1. Existing Baseline Commands

From the repository root, create/install the baseline backend environment with `make-Ctopics/degron-footprint/system/appvenv`, then run `make-Ctopics/degron-footprint/system/appdev-api`. The API listens on `http://127.0.0.1:8000`. On first run this command prepares a SQLite development database from the local research checkout; it does not start the planned PostgreSQL workspace or durable worker. The existing development ETL's default ClinVar subset must be recorded when interpreting annotations.

In a second terminal enter `topics/degron-footprint/system/app/frontend`, run `npmci`, then `npmrundev----host127.0.0.1`. Open `http://127.0.0.1:5173`. Use the real backend; mock mode contains synthetic scores. These

commands are read from the baseline guides and were not executed as application acceptance in this document-update task.

A.2.2. Target Local Workflow

[PENDING: implemented migrations and startup commands for the local PostgreSQL workspace, worker and new screens.] The final guide must reproduce reference preparation, local startup, a saved project, a peptide/protein job, a committed observation dataset, comparison/export, shutdown and reopening on the same machine. Copy database and workspace files together for recovery. Docker may package these same components; it is not a separate public/internal edition.

A.3. User Guide

The target sequence is: create/open a project; choose single peptide, peptide batch or whole protein; review syntax/domain/coverage; submit and inspect per-tile results; view ClinVar/reference evidence; import CSV/TSV with assay/units/replicates and fix validation errors; explicitly commit; compare compatible observations; export scores and provenance; reopen history. Unsaved single-tile previews are transient; saved work persists until deletion. Deletion presents dependencies and requires confirmation. **[PENDING: screenshots and an executed local walkthrough after implementation.]**

APPENDIX B: SUPPLEMENTARY MATERIALS

B.1. Source Code and Data Availability

[PENDING: repository link and release; data are redistributed only where the terms of use allow.]

B.2. Additional Tables and Figures

[PENDING]